Sveučilište u Zagrebu
Fakultet elektrotehnike i računarstva
Zavod za elektroničke sustave i obradbu informacija

Projektiranje ugradbenih računalnih sustava

Zynq - prvi susret

Mladen Vučić

Zagreb, 2022.

Sadržaj

1. Uvod 1

2. Radna okolina 2

3. Vježba 1 - Programabilna logika 3

3.1 Uvod 3
3.2 Zadatak 3
3.3 Priprema radne okoline 3
3.4 Pokretanje programa Vivado 3
3.5 Kreiranje novog projekta 4
3.6 Izrada VHDL modela digitalnog sustava 4
3.6.1 Izrada novog modela 4
3.6.2 Učitavanje prethodno razvijenih modela 5
3.7 Analiza dizajna 5
3.8 Izrada ispitnog okruženja 6
3.9 Ponašajna simulacija RTL modela 6
3.10 Sinteza 7
3.11 Definiranje zahtjeva za implementaciju 7
3.11.1 Kreiranje XDC datoteke 7
3.11.2 Definiranje priključaka 7
3.11.3 Kreiranje zahtjeva za takt 8
3.12 Implementacija 9
3.13 Generiranje datoteke za programiranje 9
3.14 Programiranje sklopa 10

4. Vježba 2 - Izrada jezgre 11

4.1 Uvod 11
4.2 Zadatak 11
4.3 Priprema radne okoline 11
4.4 Pokretanje programa Vivado 11
4.5 Kreiranje novog projekta 11
4.6 Izrada modela komponente 11
4.7 Analiza dizajna i logička sinteza 12
4.8 Izrada jezgre 12
4.8.1 Pakiranje projekta 12
4.8.2 Unos opisa jezgre 12
4.8.3 Kreiranje jezgre 13
4.9 Rezultat izrade jezgre 13

5. Vježba 3 - Dizajn digitalnog sustava pomoću jezgri 14

5.1 Uvod 14
5.2 Zadatak 14

5.3 Priprema radne okoline 14
5.4 Pokretanje programa Vivado 14
5.5 Kreiranje novog projekta 14
5.6 Uključivanje jezgre u dizajn 14
5.7 Razvoj sklopovlja 15
5.7.1 Crtanje sheme 15
5.7.2 Generiranje HDL modela pojedinih komponenata 16
5.7.3 Generiranje krovnog HDL modela 16
5.8 Analiza dizajna 16
5.9 Sinteza 16
5.10 Definiranje zahtjeva na implementaciju 17
5.10.1 Definiranje priključaka 17
5.10.2 Kreiranje zahtjeva za takt 17
5.11 Implementacija 17
5.12 Generiranje datoteke za programiranje 17
5.13 Programiranje sklopa 18
6. Vježba 4 - Ugrađeni procesor 19
6.1 Uvod 19
6.2 Zadatak 19
6.3 Priprema radne okoline 19
6.4 Pokretanje programa Vivado 19
6.5 Kreiranje novog projekta 19
6.6 Uključivanje jezgre u dizajn 20
6.7 Razvoj sklopovlja 20
6.7.1 Crtanje sheme 20
6.7.2 Generiranje HDL modela pojedinih komponenata 22
6.7.3 Generiranje krovnog HDL modela 22
6.8 Analiza dizajna 22
6.9 Sinteza 22
6.10 Definiranje zahtjeva na implementaciju 22
6.10.1 Definiranje priključaka 22
6.10.2 Kreiranje zahtjeva za takt 23
6.11 Implementacija 23
6.12 Generiranje datoteke za programiranje 23
6.13 Programiranje sklopa 23
6.14 Prijenos konfiguracije sklopovlja u programsko okruženje 23
6.15 Programska podrška temeljena na izravnom pristupu sklopovlju 24
6.15.1 Pokretanje programa Vitis 24
6.15.2 Kreiranje platforme 24
6.15.3 Dodavanje nove domene 24
6.15.4 Kreiranje prazne aplikacije 24
6.15.5 Razvoj aplikacije 25
6.15.6 Opcije za povezivanje i punjenje 25
6.15.7 Prevođenje programske podrške 25
6.16 Pokretanje i uhodavanje programske podrške 26

7. Vježba 5 - Upravljački programi 28
7.1 Uvod 28
7.2 Zadatak 28
7.3 Priprema radne okoline 28
7.4 Pokretanje programa Vivado 28
7.5 Razvoj sklopovlja 29
7.5.1 Izmjena sheme 29
7.5.2 Generiranje HDL modela pojedinih komponenata 31
7.5.3 Generiranje krovnog HDL modela 31
7.6 Analiza dizajna 31
7.7 Sinteza 31
7.8 Definiranje zahtjeva na implementaciju 31
7.9 Implementacija 32
7.10 Generiranje datoteke za programiranje 32
7.11 Prijenos konfiguracije sklopovlja u programsko okruženje 32
7.12 Programska podrška temeljena na upravljačkim programima 32
7.12.1 Pokretanje programa Vitis 32
7.12.2 Ažuriranje platforme 32
7.12.3 Ažuriranje aplikacijskog i sistemskog projekta 33
7.12.4 Izrada aplikacije 33
7.12.5 Opcije za povezivanje i punjenje 34
7.12.6 Prevođenje programske podrške 34
7.13 Uhodavanje na sustavu ZedBoard 34
7.13.1 Svjetleće diode 34
7.13.2 Serijska veza 34
7.14 Završne napomene 35
8. Vježba 6 - AXI periferije 36
8.1 Uvod 36
8.2 Zadatak 36
8.3 Priprema radne okoline 36
8.4 Pokretanje programa Vivado 36
8.5 Razvoj sklopovlja 37
8.5.1 Izmjena sheme 37
8.5.2 Generiranje HDL modela pojedinih komponenata 38
8.5.3 Generiranje krovnog HDL modela 38
8.6 Analiza dizajna 39
8.7 Sinteza 39
8.8 Definiranje zahtjeva na implementaciju 39
8.9 Implementacija 39
8.10 Generiranje datoteke za programiranje 39
8.11 Prijenos konfiguracije sklopovlja u programsko okruženje 39
8.12 Programska podrška 40
8.12.1 Pokretanje programa Vitis 40
8.12.2 Ažuriranje platforme 40
8.12.3 Ažuriranje aplikacijskog i sistemskog projekta 40

8.12.4 Prepravljanje aplikacije 40
8.12.5 Prevođenje programske podrške 40
8.13 Uhodavanje na sustavu ZedBoard 40
9. Savjeti za rad s alatima 41
9.1 Podešavanje opcija alata 41
9.1.1 Vivado 41
9.1.2 Vitis 41
9.2 Imena datoteka i duljina puta 41
9.3 Primjer naziva datoteka 41
9.3.1 Vivado 42
9.3.2 Vitis 42
9.4 Položaj datoteka 42
9.5 Prijenos konfiguracije sklopovlja u programsko okruženje 43
9.6 Promjena sklopovlja računalnog sustava 43
9.6.1 Ažuriranje platforme 43
9.6.2 Ažuriranje aplikacijskog i sistemskog projekta 43
9.7 Premještanje radne mape projekta 44
9.7.1 Vivado 44
9.7.2 Vitis 44
9.8 Kopiranje aplikacijskog projekta u drugo radno okruženje 44
9.9 Sinteza i definiranje priključaka 44
9.10 XDC datoteka 45
9.11 Postupak prevođenja programske podrške 45
9.12 Resetiranje projekta 45
10. Završne napomene 46

1. Uvod

Ova skripta pisana je kao podrška materijalima za predavanja predmeta *Projektiranje gradbenih računalnih sustava*. Služi kao priručnik za samostalan rad studenata kod kuće i u laboratoriju. Obuhvaća prvu nastavnu cjelinu.

Opisane vježbe daju praktična znanja za rad sa *System on Chip* (SoC) komponentama familije Zynq-7000, proizvođača Xilinx Inc. U tom smislu, vježbe imaju tri cilja. Prvi cilj je upoznavanje s građom sustava Zynq. Drugi cilj je upoznavanja s alatima koji se koriste za razvoj sklopova i programske podrške. Treći cilj je savladavanje koncepata karakterističnih za razvoj računalnih sustava u SoC okruženju.

Vježbe obuhvaćaju sljedeće teme:

- razvoj sklopovlja namijenjen implementaciji u programabilnoj logici
- generiranje IP jezgara i njihovo korištenje u blokovskom dizajnu
- rad s ugrađenim procesorom
- upoznavanje s periferijama i načinom njihovog programiranja
- korištenje upravljačkih programa i jednostavnih operacijskih sustava
- povezivanje procesorskog sustava i programabilne logike

Napomena:

Za uspješno savladavanje vježbi potrebno je poznavati gradivo obrađeno na predavanjima. Zato je prije svake vježbe potrebno ponoviti određena poglavlja iz materijala za predavanja.

Velik dio rješenja zadataka može se pronaći u materijalima za predavanjima. Međutim, preporučuje se ne kopirati pojedine dijelove koda već ih napisati samostalno i s razumijevanjem. Uz takav način rada moguće je bolje savladati gradivo te kasnije samostalno rješavati slične zadatke.

2. Radna okolina

Razvoj računalnog sustava koji se temelji na SoC platformi obuhvaća

- razvoj sklopovlja
- razvoj programske podrške

Tijekom razvoja bit će korišteno integrirano razvojno okruženje koje sadrži dva alata

- Vivado
- Vitis

Prvi alat namijenjen je razvoju sklopova u FPGA okruženju, dok drugi služi za razvoj programske podrške za ugrađene mikrokontrolere s ARM procesorskom jezgrom.

Vježbe obuhvaćaju pripremu kod kuće te rad u laboratoriju. U okviru pripreme potrebno je razviti sklopovlje i programsku podršku te ih uhodati u simulatoru koliko je to moguće. U laboratoriju je potrebno ispitati rad sklopovskih i programskih komponenata koje su razvijene kod kuće te ih uhodati na stvarnom sklopovlju.

Za rad kod kuće potrebno je na vlastita računala instalirati programske pakete Xilinx Vitis 2020.2 i Vivado 2020.2. Upute za instalaciju navedenih paketa nalaze se u zasebnim dokumentima za webu predmeta.

Kao sklopovska platforma odabran je razvojni sustav *ZedBoard - Zynq Evaluation and Development Kit*. Ovaj sustav je studentima na raspolaganju za vrijeme rada u laboratoriju.

3. Vježba 1 - Programabilna logika

3.1 Uvod

Ova vježba ilustrira faze dizajna digitalnog sustava pomoću jezika za opis sklopovlja. Dizajn se provodi u programskom paketu Vivado. Koristi se isključivo programabilna logika, a ne koristi se procesorski sustav.

U vježbi će biti razvijena i uhodana komponenta *led_blinker* koja će u narednim poglavljima biti spakirana u IP jezgru te dalje korištena kao jezgra.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeća poglavlja iz materijala za predavanja:

3 SoC Zynq-7000
4 Električko spajanje SoC sklopa
5 Dizajn temeljen na sklopovskim jezgrama
do uključivo ***5.7 Implementacija na sustavu ZedBoard***

3.2 Zadatak

Potrebno je razviti VHDL model sklopa koji na svom izlazu daje pravokutni signal perioda između 1 s i 2 s. Pretpostaviti da sklop radi na taktu frekvencije 100 MHz. Sklop mora imati reset ulaz čije aktivno stanje na izlazu daje logičku 0. Osigurati ispravan reset pod pretpostavkom da do njega ne dolazi sinkrono s taktom. Entitet sklopa nazvati *led_blinker*. Jedna od komponenata ovog entiteta bit će sinkrono brojilo, za koje je potrebno predvidjeti generičku duljinu. Generičku konstantu koja određuje broj bitova brojila nazvati *CLK_divider_width* te ju pohraniti u paket *init01*.

Razvijeni sklop upotrijebiti u krovnom modelu koji u protufazi pali i gasi dvije svjetleće diode. Reset signal dovesti s tipke. Krovni model nazvati *all_design*.

Implementirati krovni model na sustavu ZedBoard. Signale za pogon svjetlećih dioda spojiti na diode LD0 i LD1. Signal za reset dovesti s tipke BTND.

3.3 Priprema radne okoline

Potrebno je napraviti mapu

prezime_ime

koji će sadržavati rješenja pojedinih zadataka. U imenu mape ne koristiti bjeline (*space*) već znak '_'. Sve datoteke koje pripadaju pojedinoj vježbi bit će pohranjene u odgovarajuću podmapu mape **prezime_ime**. U skladu s tim, za ovu vježbu potrebno je napraviti mapu

prezime_ime\v1

3.4 Pokretanje programa Vivado

Program Vivado pokreće se iz odabirom komande

Start/Xilinx Design Tools/Vivado 2020.2

3.5 Kreiranje novog projekta

Odabrati komandu

File/Project/New

čime se pokreće čarobnjak za izradu novog projekta.

U kartici *Project Name* upisati ime projekta i odabrati željenu mapu u kojem će se projekt nalaziti. U ovom slučaju, mapa je *prezime_ime\v1*. S obzirom da alat tijekom svog rada kreira stabla s dugim putovima, koja ponekad mogu prijeći dopuštenih 256 znakova, preporučljivo je odabrati kratko ime projekta, npr. *p*. Isključiti opciju *Create project subdirectory*, te odabrati *Next*.

U kartici *Project Type* odabrati RTL projekt i uključiti opciju *Do not specify sources at this time*. Odabrati *Next*.

U kartici *Default Part* odabrati dio koji sadrži razvojne sustave (*Boards*). Pronaći i odabrati *ZedBoard Zynq Evaluation and Development Kit*, te odabrati *Next*. Odabrati *Finish*.

U prozoru *Flow Navigator* odabrati

Project Manager/Settings

i u skupu postavki *General* podesiti VHDL kao ciljani jezik (*Target Language*). To treba učiniti u ovom trenutku jer neke verzije alata ne dopuštaju da se ovo napravi kasnije.

Novi projekt je sada napravljen. U budućim pokretanjima Vivada ovaj projekt moguće je otvoriti ili zatvoriti pomoću komandi

File/Project/Open

odnosno

File/Close Project

3.6 Izrada VHDL modela digitalnog sustava

Izrada VHDL modela digitalnih sustava, pripadajućih ispitnih okruženja i sl. izvodi se u skupu prozora pod imenom *Project Manager*. Ovaj skup prozora može se uvijek učiniti vidljivim pomoću komande

Flow/Project Manager

Pregled datoteka s izvornim kodom vidljiv je u prozoru *Sources* (gore lijevo u prozoru *Project Manager*). Uz odabran *Project Manager*, ovaj prozor uvijek se može učiniti vidljivim pomoću komande

Window/Sources

3.6.1 Izrada novog modela

U vježbi će biti potrebno razviti modele:

led_blinker.vhd
all_design.vhd

te paket

init01.vhd

Ove modele kreira se i unosi na način opisan u daljnjem tekstu.

Prvo će biti razvijen model *led_blinker*. U prozoru *Sources* pomoću desnog gumba miša odabrati *Add Sources*. Odabrati *Add or Create Design Sources*, te odabrati *Next*. Odabrati tipku *Create File*. Odabrati jezik VHDL, za ime datoteke upisati *led_blinker*, ostaviti ponuđenu lokaciju, te odabrati *OK*. Odabrati *Finish*. Prozor *Define Module* preskočiti pomoću *Cancel*.

U prozoru *Sources* odabrati novonastalu datoteku *led_blinker.vhd*. Pomoću desnog gumba miša odabrati *Open File*, čime se otvara prozor koji prikazuje sadržaj datoteke, a koji je još uvijek prazan. U ovaj prozor upisati željeni VHDL kod entiteta. Prema potrebi poslužiti se rješenjem primjera koji je obrađen na predavanjima. Ne zaboraviti pohraniti datoteku.

Na opisan način sada je potrebno napraviti model *all_design*. Ovaj model bit će najviši u hijerarhiji. Nakon što je pisanje modela završeno, potrebno je pohraniti ga. Ako su modeli ispravno napisani, u prozoru *Sources* oni su poredani na način koji određuje njihova hijerarhija.

Komponenta *led_blinker* je generička komponenta. Vrijednost generičke konstante *CLK_divider_width* bit će zadana u paketu *init01*. Sada je potrebno napraviti ovaj paket. S obzirom da je paket također VHDL model, potrebno ga je napraviti slijedeći istu proceduru kao i kod kreiranja modela *led_blinker* i *all_design*.

3.6.2 Učitavanje prethodno razvijenih modela

Modeli koji su ranije razvijeni u okviru nekog drugog projekta mogu se učitati u aktivan projekt. Pritom je potrebno učiniti vidljivim prozor *Project Manager* i u njemu prozor *Sources*. Pomoću desnog gumba miša potrebno je odabrati *Add Sources*. Odabrati *Add or Create Design Sources*, te odabrati *Next*. U ovom slučaju, potrebno je odabrati tipku *Add Files*, pronaći željenu datoteku te odabrati *Finish*.

U ovom primjeru, opisana operacija neće biti potrebna.

3.7 Analiza dizajna

Nakon što su modeli razvijeni, potrebno ih je napraviti vizualnu analizu. Opcije ove analize postavljaju se pomoću komande

Flow/Settings/Elaboration Settings

U ovom primjeru potrebno je ostaviti opcije u *default* stanju.

Odabrati komande

Flow/Open Elaborated Design
Layout/Default Layout

Za prikaz sheme, u prozoru *Flow Navigator* (lijevo) odabrati komandu

RTL Analysis/Open Elaborated Design/Schematic

Za komponente iz shematskog prikaza može se lako pronaći njihov izvorni kod (*definition*) ili dio koda u kojem je spojena sama jedinka (*instance*). Za prikaz koda potrebno

je odabrati komponentu u shematskom prikazu te pomoću desnog gumba miša odabrati jednu od komandi

Go to Source

ili

Go to Definition

Proučiti dobivenu shemu i provjeriti jesu li sve komponente spojene onako kako je zamišljeno.

3.8 Izrada ispitnog okruženja

Nakon što je napravljena vizualna provjera dizajna, potrebno je provesti funkcijsku simulaciju modela. Za ovu simulaciju, potrebno je razviti ispitno okruženje (*test bench*). U prozoru *Sources* pomoću desnog gumba miša odabrati *Add Sources*. Odabrati *Add or Create Simulation Sources*, te odabrati *Next*. Odabrati *Create File*. Odabrati jezik VHDL i ime datoteke. Uobičajeno je ispitnom okruženju dati ime prema entitetu za kojeg ispituje, uz dodatak *_tb* na kraju imena. U ovom slučaju, simulacija će ispitati cjelokupno sklopovlje pa će ispitno okruženje biti predstavljeno modelom *all_design_tb*. Ne mijenjati ponuđenu lokaciju, te odabrati *OK*. Odabrati *Finish*. Prozor *Define Module* preskočiti pomoću *Cancel*.

Sada je potrebno razviti VHDL kod ispitnog okruženja. U prozoru *Sources* odabrati novonastalu datoteku *all_design_tb.vhd*. Pomoću desnog gumba miša odabrati *Open File*, čime se otvara prozor koji prikazuje sadržaj datoteke. Upisati u prozor VHDL kod. Po potrebi poslužiti se primjerom s predavanja. Pohraniti datoteku.

3.9 Ponašajna simulacija RTL modela

Ponašajnu (*behavioral*) simulaciju je moguće provesti ugrađenim (Vivado) ili vanjskim simulatorom (npr. Questa). Ovdje će biti opisano pokretanje ugrađenog simulatora.

Odabrati komandu

Flow/Settings/Simmulation Settings

U ovom prozoru zadaju se parametri simulacije. Potrebno je odabrati *Vivado Simulator*, te skup modela koji se koriste za simulaciju, odnosno ispitno okruženje *all_design_tb.vhd* razvijeno u prethodnom poglavlju. Na kartici *Simulation* postaviti vrijeme simulacije, *xsim.simulation.runtime*, na 1000 ns. Ostale parametre ostaviti nepromijenjene i odabrati *OK*.

U paketu *init01* postaviti vrijednost konstante *CLK_divider_width* na 3. Brojilo od tri bita dat će zorniji prikaz rada sklopa, a simulacija će trajati prihvatljivo vrijeme.

Odabrati komandu

Flow/Run Simulation/Run Behavioral Simmulation

Nakon što simulacija završi valni oblici signala bit će prikazani u prozoru. Proučiti valne oblike i provjeriti ispravnost rada sustava. Uočiti da su aktivni prozori sada objedinjeni u skupu koji se naziva *Behavioral Simulation*.

3.10 Sinteza

U paketu *init02* postaviti vrijednost konstante *CLK_divider_width* na 26.

Opcije sinteze postavljaju se pomoću komande

Flow/Settings/Synthesis Settings

U ovom primjeru potrebno je ostaviti opcije u *default* stanju.

Pokrenuti sintezu pomoću komande

Flow/Run Synthesis

Nakon što sinteza završi, preskočiti pomoću *Cancel* akcije koje alat eventualno ponudi u prozoru *Synthesis Completed*. Napomena: sinteza može potrajati neko značajnije vrijeme.

Izvještaj o sintezi može se dobiti pomoću komande

Window/Reports

Ova komanda čini vidljivim prozor *Reports* koji sadrži sve izvještaje.

U prozoru *Flow Navigator* (lijevo) moguće je odabrati komandu

Synthesis/Synthesized Design/Schematic

kojom se prikazuje sheme dobivena sintezom. Pokrenuti ovu komandu i proučiti dobivenu shemu.

3.11 Definiranje zahtjeva za implementaciju

Prije implementacije potrebno je definirati zahtjeve postavljene na takt, priključke i sl. Ovi zahtjevi mogu se opisati na više načina. U ovom primjeru oni će biti zadani pomoću datoteke *Xilinx Design Constraints*, koja ima nastavak *.xdc*.

3.11.1 Kreiranje XDC datoteke

U prozoru *Sources* pomoću desnog gumba miša odabrati *Add Sources*. Odabrati *Add or Create Constraints*, te odabrati *Next*. Odabrati *Create File*. Odabrati tip datoteke XDC, ime datoteke koje u ovom slučaju neka bude *all_design*, ostaviti ponuđenu lokaciju, te odabrati *OK*. Odabrati *Finish*.

3.11.2 Definiranje priključaka

Uz pretpostavku da je sinteza već napravljena, pokrenuti komande

Flow/Open Synthesized Design
Window/ I/O Ports

Dobiven prozor *I/O Ports* sadrži sve priključke entiteta *all_design*. (Prema potrebi prozor se može maksimizirati.) Pojedinim priključcima sada je potrebno dodijeliti odgovarajuće parametre. Pod pretpostavkom da će sklop biti implementiran na razvojnom sustavu ZedBoard, parametri imaju sljedeće vrijednosti

- clk
  - priključak (*Site*) Y9, banka (*Bank*) 13, *Fixed*
  - I/O Standard LVCMOS33
  - (NAPOMENA: banka 13 fiksno je spojena na 3.3V)
- led0
  - priključak (*Site*) T22, banka (*Bank*) 33, *Fixed*
  - I/O Standard LVCMOS33
  - Drive Strength 12
  - Slew Type Slow (None)
  - Pull Type None
  - Off-Chip Termination None
  - (NAPOMENA: banka 33 fiksno je spojena na 3.3V)
- led1
  - priključak (*Site*) T21, banka (*Bank*) 33, *Fixed*
  - I/O Standard LVCMOS33
  - Drive Strength 12
  - Slew Type Slow (None)
  - Pull Type None
  - Off-Chip Termination None
  - (NAPOMENA: banka 33 fiksno je spojena na 3.3V)
- reset
  - priključak (*Site*) R16, banka (*Bank*) 34, *Fixed* (tipka BTND)
  - I/O Standard LVCMOS25
  - (NAPOMENA: Banka 34 spojena je na podesivo napajanje. Na sustavu ZedBoard potrebno je spojiti J18 na 2.5V)

Usporediti ove parametre s tablicama u poglavlju *5.7 Implementacija na sustavu ZedBoard* iz materijala za predavanja.

Nakon što su priključcima dodijeljeni parametri, potrebno je parametre pohraniti u datoteku *all_design.xdc*. Pohrana se izvodi pomoću komande

File/Constraints/Save

U prozoru *Sources* odabrati datoteku *all_design_xdc*. Pomoću desnog gumba miša odabrati komandu *Open File* i tako otvoriti datoteku. Proučiti njen sadržaj.

3.11.3 Kreiranje zahtjeva za takt

U prozoru *Flow Navigator* odabrati komandu

Synthesis/Synthesized Design/Edit Timing Constraints

Nakon pojave prozora (kartice) *Timing Constraints*, u prozoru koji sadrži stablo odabrati pomoću desnog gumba miša

Create Timing Constraints/Clock/Create Clock

Nakon pojave prozora *Create Clock*, upisati ime takta, *Clock Name*. Preporučljivo je da ovo ime odgovara imenu signala, u ovom slučaju *clk*. Za *Source Object* odabrati '**...**' . Odabrati *I/O Port* s imenom (*name*) *clk* u ulaznim priključcima (*in*), te odabrati *Find*. Pronađeni takt prebaciti u prozor *Selected names* te odabrati *Set* i *OK*. Podesiti parametre *Period*, *Rise at* i *Fall at*. Upisati vrijednosti 10 ns, 0 ns i 5 ns, te odabrati *OK*.

Pohraniti u datoteku *all_design.xdc*. Pohrana se izvodi pomoću komande

File/Constraints/Save

u postojeću XDC datoteku (*Select an existing file*).

Ponovo učitati (*reload*) datoteku *all_design.xdc*, te prema potrebi ručno urediti njen sadržaj.

3.12 Implementacija

Postavke sinteze zadaju se pomoću komande

Flow/Settings/Implementation Settings

Provjeriti u postavkama je li aktivan ispravan skup uvjeta *Constraints* i po potrebi promijeniti skup.

U paketu *init01* postaviti vrijednost konstante *CLK_divider_width* na 26.

Pokrenuti implementaciju pomoću komande

Flow/Run Implementation

Po potrebi ponoviti i ranije napravljenu sintezu. Nakon što implementacija završi, preskočiti pomoću *Cancel* akcije koje alat eventualno ponudi u prozoru *Implementation Completed*. Napomena: implementacija može potrajati neko značajnije vrijeme.

Izvještaj o implementaciji može se dobiti pomoću komande

Window/Reports

Njeno pokretanje čini vidljivim prozor *Reports* koji sadrži sve izvještaje. Otvoriti izvještaj *Timing Summary Report* i provjeriti jesu li zadovoljeni svi zahtjevi.

3.13 Generiranje datoteke za programiranje

Parametri generatora datoteke za programiranje FPGA sklopa (*Bitstream*) podešavaju se pomoću komande.

Flow/Settings/Bitstream Settings

U ovom primjeru nije potrebno mijenjati ove parametre. Izrada datoteke za programiranje pokreće se komandom

Flow/Generate Bitstream

Nakon što generiranje završi, preskočiti pomoću *Cancel* akcije koje alat eventualno ponudi u prozoru *Bitstream Generation Completed*.

3.14 Programiranje sklopa

Provjeriti jesu li premosnici podešeni na ispravan način. Posebno provjeriti premosnik J18.

Dovesti napajanje na razvojni sustav ZedBoard. Povezati sustav i osobno računalo pomoću USB kabela, preko priključka PROG. Uključiti napajanje tipkom POWER.

Za programiranje se koristi alat iMPACT ili neki drugi njemu srodan alat. Alat se pokreće pomoću komande

Flow/Open Hardware Manager

Ako se u novootvorenom prozoru nije pojavio JTAG lanac, pokrenuti *Open Target/Auto Connect*, koja je vidljiva u dijelu *Flow Navigator/Program and Debug*.

Programiranje se izvodi pomoću komande *Program Device/XC7Z020_1*.

4. Vježba 2 - Izrada jezgre

4.1 Uvod

Ovo poglavlje opisuje izradu jezgre koja ne sadrži AXI sučelje. Kao primjer napravljena je jezgra *led_blinker*, čiji model je razvijen vježbi 1. Iako je jezgru moguće napraviti iz projekta opisanog u vježbi 1, ona će ovdje radi zornije ilustracije postupka biti napravljena u zasebnom projektu.

Cilj vježbe je upoznati čitatelja s konceptom IP jezgre. Zato će jezgra biti napravljena u svom najjednostavnijem obliku. U dizajnu koji je namijenjen tržištu jezgri, samoj jezgri bile bi dodane mnoge opcije i mogućnosti o kojima ovdje zbog nedostatka vremena neće biti govora.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeće poglavlje iz materijala za predavanja:

5.8 Kreiranje jezgri

4.2 Zadatak

Sklop *led_blinker* koji je razvijen u vježbi 1 potrebno je realizirati u obliku IP jezgre.

4.3 Priprema radne okoline

Sve datoteke koje pripadaju ovoj vježbi bit će pohranjene u zasebnu podmapu mape **prezime_ime**. U skladu s tim, za ovu vježbu potrebno je napraviti mapu

prezime_ime\v2

4.4 Pokretanje programa Vivado

Pokrenuti program Vivado. Ako se je pritom automatski učitao neki projekt, zatvoriti ga pomoću komande

File/Close Project

4.5 Kreiranje novog projekta

Na način opisan u poglavlju 3.5 kreirati novi projekt. Projekt je potrebno pohraniti u mapu *prezime_ime\v2*.

4.6 Izrada modela komponente

Na način opisan u poglavlju 3.6 napraviti model *led_blinker*. To je najlakše učiniti kopiranjem koda iz datoteke *led_blinker.vhd* iz vježbe 1. Spomenuti VHDL model nalaze se u mapi

prezime_ime\v1\p.srcs\sources_1\new

Napomena:

Datoteku je moguće dodati i na način opisan u poglavlju 3.6.2. U tom slučaju mapa *prezime_ime\v2* ne bi sadržavala sve datoteka ove vježbe (*led_blinker.vhd* bio bi i dalje u *prezime_ime\v1*). To bi moglo uzrokovati poteškoće prilikom prijenosa projekta na drugo računalo.

S obzirom da će jedino ovaj entitet biti pakiran u IP jezgru, nije potrebno napraviti entitet *all_design* niti paket *init01*.

4.7 Analiza dizajna i logička sinteza

Prema uputama u poglavljima 3.7 i 3.10 provesti analizu dizajna te logičku sintezu.

4.8 Izrada jezgre

4.8.1 Pakiranje projekta

Ako se pakiranje ne radi prvi put, potrebno je obrisati datoteku *component.xml*. Ona je vidljiva u prozoru *Sources*, u dijelu *Design Sources/IP-XACT*.

Otvoriti rezultat sinteze (*Open Synthesized Design*). Odabrati komandu

Tools/Create and Package IP

te prozoru *Create and Package New IP* odabrati *Next*. U prozoru *Create Peripheral, Package IP or Package Block Design* odabrati *Package your current project*. NE uključivati opciju *Create a new AXI4 Peripheral*. Odabrati *Next*. U prozoru *Package Your Current Project* podesiti *IP Location*. Pritom odabrati radnu mapu koju ponudi alat (*prezime_ime/v2/p.srcs/sources_1/new*). Ako se odabere druga mapa, može se pojaviti *warning* koji kaže da putovi nisu relativni. Putove koji nisu relativni treba izbjegavati.

Odabrati *Next* i *Finish*. Ako se pojavi prozor *IP Packager Summary*, proučiti ga i pritisnuti *OK*.

4.8.2 Unos opisa jezgre

Opis jezgre nalazi se u prozoru *Sources*, dijelu *IP_XACT*, u datoteci *component.xml*. Otvaranjem ove datoteke pojavljuje se prozor (kartica) *Package IP - led_blinker* u kojem kroz više koraka može promijeniti opis jezgre. Odabrati korak (*Packaging Step*) *Identification* i popuniti polja. Preporučuje se u ovom primjeru samo promijeniti ime ponuđača jezgre (*Vendor*) u ESDL. (Ovaj naziv dolazi od engleskoga naziva vježbi predmeta, *Embedded System Design Laboratory*.) U polje *Categories* unosi se popis kategorija odnosno područja primjene kojima jezgra pripada. Kategorija se dodaje komandom *Add* ili tipkom +. Nakon pojave popisa, odabrati kategoriju koja najbolje opisuje jezgru. U ovom slučaju to je *UserIP*.

Odabrati korak *Compatibility* i prema potrebi podesiti familiju sklopova na kojima će ova jezgra moći biti korištena. U ovom primjeru nije potrebno izbacivati niti jednu familiju.

Odabrati korak *File Groups*. Proučiti datoteke koje su dio jezgre i njihov položaj u hijerarhiji. U ovom primjeru nije potrebno mijenjati ove parametre.

Odabrati korak *Customization Parameters*. Uočiti da se u ovom koraku opisuju parametri koji služe za konfiguraciju jezgre. Ovi parametri mogu se automatski preuzeti pomoću čarobnjaka (*wizard*) iz entiteta *led_blinker* ili se mogu editirati ručno pomoću

komande *Edit Parameters* koja se dobiva pomoću desnog gumba miša. Podesiti minimalnu i maksimalnu širinu brojila n 2 odnosno 32 bita

Odabrati korak *Ports and Interfaces*. U ovom koraku opisuju se parametri priključaka kao što je smjer, tip i sl. Ovi parametri dobiveni su automatski iz entiteta *led_blinker*, a mogu se mijenjati i ručno. Postaviti miša na traku s opisom parametara (siva traka) i pomoću desno gumba odabrati vidljivost željenih parametara. Uočiti da je IP pakiranje automatski uključilo signale *clock* i *reset* u grupe koje predstavljaju sučelja (*Bus Interfaces*). Npr., *reset* se nalazi u grupi *reset*. Do toga dolazi jer većina jezgri ima sučelja (npr. AXI). Ova sučelja koristi sustav za spajanje jezgri (*Block Integrator*) kod provjere ispravnosti spajanja.

Odabrati korak *Customization GUI*. Provjeriti je li oblik i polaritet priključaka dobar.

Napomene:

Ako u GUI jezgre neki od priključaka ima pogrešan polaritet, npr. kružić na ulazu na kojem ne bi trebao postojati, vratiti se na korak *Ports and Interfaces*. Odabrati sučelje (*Bus Interface*) koji odgovara tom signalu i izbrisati ga pomoću komande *Remove Interface* (dobiva se desnim gumbom miša). Isto učiniti ako se pojavi greška oblika *[IP_Flow 19-5661] Bus Interface 'clk' does not have any bus interfaces associated with it*.

Kod ovako jednostavnih sklopova može se (i preporučljivo je) obrisati sva sučelja u koje su grupirani signali. Ako se to učini, u prozoru će biti vidljivi samo signali.

4.8.3 Kreiranje jezgre

Odabrati *Review and Package* i pokrenuti *Edit packaging settings*. U kartici *Packager* uključiti u *After Packaging* opciju *Create archive of IP*, da se generira *.zip* file. Ostale opcije postaviti po želji te odabrati *Apply* i *OK*. Pritisnuti tipku *Package IP*.

4.9 Rezultat izrade jezgre

Rezultat izrade jezgre je *.zip* datoteka koja se pojavila u radnom direktoriju. U ovom slučaju to je datoteka *ESDL_user_led_blinker_1.0.zip*. Ona sadrži direktoriji *xgui* datoteke *led_blinker_0* i *component.xml*.

Spomenuta .zip datoteka sadrži sve informacije o dizajnu i isporučuje se korisniku (kupcu) kao jezgra. Ona se može staviti u direktorij u kojem se nalaze druge jezgre. Taj direktorij može se uključiti u radnu okolinu, što će biti ilustrirano u sljedećoj vježbi.

5. Vježba 3 - Dizajn digitalnog sustava pomoću jezgri

5.1 Uvod

Ovo poglavlje opisuje dizajn sustava temeljen na prethodno napravljenim ili kupljenim jezgrama. Ovakav dizajn karakterističan je za Vivado. U njemu se digitalni sustav oblikuje kroz takozvani blokovski dizajn (*block design*), koji se izvodi u grafičkom sučelju. Za ilustraciju takvog pristupa, ovdje će biti korištena jezgra koja je napravljena u vježbi 2.

U primjeru će biti korištena isključivo programabilna logika. Jezgra će biti spojena na vanjski takt koji dolazi na PL podsustav.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeće poglavlje iz materijala za predavanja:

5.9 Razvoj sklopovlja temeljen na jezgrama

5.2 Zadatak

Korištenjem jezgre razvijene u vježbi 2 razviti sklopovlje koji na razvojnom sustavu ZedBoard pali i gasi dvije svjetleće diode LD2 i LD3. Dioda LD2 treba se paliti i gasiti s periodom između 1 s i 2 s. Dioda LD3 treba se paliti i gasiti četiri puta sporije.

Svaku diodu neka pogoni njena vlastita jezgra. Jezgre trebaju raditi s vanjskim izvorom takta frekvencije 100 MHz. Za reset sklopovlja upotrijebiti tipku BTNC.

5.3 Priprema radne okoline

Sve datoteke koje pripadaju ovoj vježbi bit će pohranjene u zasebnu podmapu mape **prezime_ime**. U skladu s tim, za ovu vježbu potrebno je napraviti mapu

prezime_ime\v3

5.4 Pokretanje programa Vivado

Pokrenuti program Vivado. Ako se je pritom automatski učitao neki projekt, zatvoriti ga pomoću komande

File/Close Project

5.5 Kreiranje novog projekta

Na način opisan u poglavlju 3.5 kreirati novi projekt. Projekt je potrebno pohraniti u mapu *prezime_ime\v3*.

5.6 Uključivanje jezgre u dizajn

U velikim projektima nova jezgra stavlja se u repozitorij u kojem se nalaze i druge jezgre. Međutim, ovdje će jezgra zbog kompaktnosti primjera biti pohranjena u radnu mapu projekta. U mapi *prezime_ime\v3* kreirati mapu *my_cores* i presnimiti u nju arhiviranu jezgru *ESDL_user_led_blinker_1.0.zip* koja je napravljena u vježbi 2.

Jezgra s u IP katalog uključuje pokretanjem komande

Project Manager/IP Catalog

Nakon pokretanja ove komande, na ekranu se prikazuje katalog koji sadrži jezgre koje proizvodi ili prodaje proizvođač FPGA i SoC sklopova. Neke od ovih jezgri smiju se koristiti besplatno, dok su neke podložne licenciranju.

Desnim gumbom miša pokrenuti komandu *Add Repository* i odabrati mapu *my_cores*, u koju je pohranjena datoteka *ESDL_user_led_blinker_1.0.zip*, te dodati mapu u repozitorij. Odabrati dodanu mapu i pokrenuti komandu *Add IP to Repository*, odabrati našu *.zip* datoteku te odabrati *OK*. Jezgra je sada vidljiva blokovskom dizajnu i može se uključiti u shemu pomoću komande *Add IP*.

5.7 Razvoj sklopovlja

5.7.1 Crtanje sheme

Pokrenuti komandu

Flow/Create Block Design

Upisati ime koje se želi dati dizajnu (*Design Name*). Preporučuje se odabrati ime *v3* te odabrati *OK*. Nakon nekog vremena pojavljuje se prozor naziva *Diagram* koji će sadržavati shemu sklopa. Ovaj prozor može se kasnije otvarati pomoću komande

Flow/Open Block Design

U prozoru *Diagram* pomoću desne tipke miša odabrati *Add IP*, te odabrati jezgru *led_blinker*. Nakon pojave jezgre na shemi, odabrati ju (na shemi) i pomoću desne tipke miša odabrati *Customize Block*. Podesiti parametar *CLK Divider Width* na odgovarajuću vrijednost. Prema potrebi, poslužiti se primjerom s predavanja. Dodati na shemu još jednu jezgru te podesiti njen parametar *CLK Divider Width* (paziti: ne radi se o istoj vrijednosti).

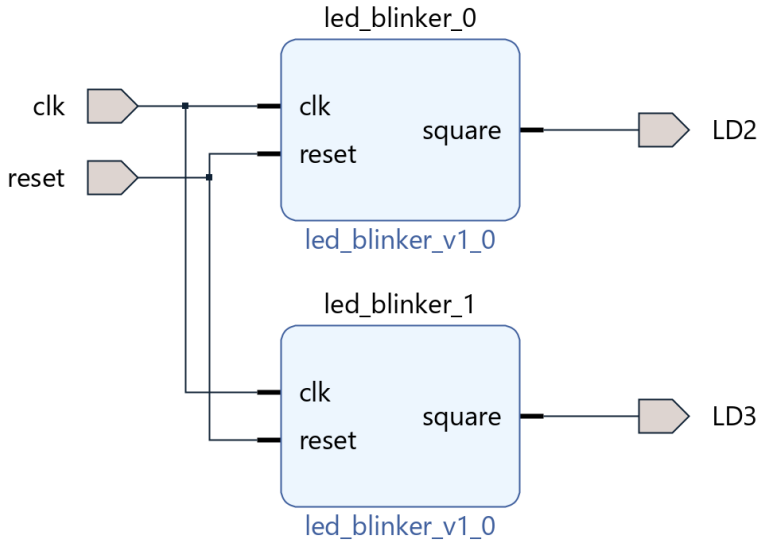
U ovom primjeru izlazi komponenata *led_blinker* izravno pogone svjetleće diode. Ulazi u komponente su takt i reset. Oba ova signala dolaze s vanjskih priključaka sklopa Zynq. Prvi signal dolazi s namjenskog priključka za takt, a drugi s ulazno-izlaznog priključka na koji je spojena tipka. U ovoj fazi izrade sheme spojit ćemo priključke komponente na vanjske priključke krovnog entiteta. Spajanje se izvodi pomoću komande *Make External* ili crtanjem žica.

Desnim gumbom miša selektirati jedan po jedan vanjski priključak i pomoću komande *External Port Properties* promijeniti imena priključaka da odgovaraju onima u zadatku. U konačnici, shema mora imati oblik prikazan slikom 5.1.

Prema potrebi, u prozoru *Diagram* pomoću desne tipke miša pokrenuti *Regenerate Layout*. Ova komanda uljepša shemu.

U prozoru *Diagram* pomoću desne tipke miša pokrenuti *Validate Design* i prema potrebi ispraviti greške.

Gotov dizajn sprema se pomoću komande *File/Save Block Design*.



Slika 5.1 Shema dobivana blokovskim dizajnom.

5.7.2 Generiranje HDL modela pojedinih komponenata

U prozoru *Flow Navigator* odabrati *Settings* te u kartici *General* podesiti VHDL kao ciljani jezik (*Target Language*). Odabrati *Apply* i *OK*.

U prozoru *Sources* odabrati najviši sustav u hijerarhiji (*.bd*) i pomoću desne tipke miša pokrenuti *Reset Output Product* te odabrati *Reset*. Ova operacija nije potrebna ako *Output product* već ne postoji od ranije. Zatim pokrenuti *Generate Output Products* te odabrati *OK*.

5.7.3 Generiranje krovnog HDL modela

U prozoru *Sources* odabrati najviši sustav u hijerarhiji i pomoću desne tipke miša pokrenuti *Create HDL Wrapper*. Ovdje je moguće prepustiti Vivadu brigu o krovnom modelu (*Let Vivado manage wrapper and auto update*) ili omogućiti korisniku da intervenira u njega (*Copy generated wrapper to allow user edit*). U prvom slučaju postoji samo jedna kopija krovnog modela koja se nalazi u direktoriju *Design Sources*. U drugom slučaju postojala bi i njena kopija u direktoriju *imports*. Sve ove datoteke na disku se nalaze u mapi koja sadrži izvorni kod, čije ima nastavak *.src*. U ovoj fazi učenja alata bolje je odabrati drugu opciju.

Ako su sve gornje operacije ispravno napravljene, dobiva se model *v3_wrapper.vhd*. Otvoriti ovu datoteku i proučiti njen sadržaj.

5.8 Analiza dizajna

Provesti analizu dizajna na način opisan u poglavlju 3.7. Analizirati dobivenu shemu, provjeriti je li ona u skladu s očekivanjima i po potrebi ispraviti greške.

5.9 Sinteza

Provesti sintezu na način opisan u poglavlju 3.10.

5.10 Definiranje zahtjeva na implementaciju

5.10.1 Definiranje priključaka

Definiranje zahtjeva na implementaciju opisano u poglavlju 3.11. Ovdje je potrebno kreirati datoteku *v3.xdc* (vidi poglavlje 3.11.1) i definirati priključke (vidi poglavlje 3.11.2).

U ovom primjeru potrebno je definirati priključke za četiri signala. Dva signal su ulazna,

- clk
- reset

a dva izlazna,

- LD2
- LD3

Za sve signale potrebno je definirati:

- priključak
- IO Standard

Za izlazne priključke potrebno je definirati još i

- Drive Strength
- Pull Type
- Off-Chip Termination

Za odabir ovih parametara poslužiti se podacima danim na predavanju ili priručnikom za rad s razvojnim sustavom ZedBoard.

Nakon što su priključcima dodijeljeni parametri, ne zaboraviti parametre pohraniti u datoteku *v3.xdc* pomoću komande

File/Constraints/Save

Nakon definiranja priključaka otvoriti datoteku *v3.xcd* i proučiti njen sadržaj. Sadržaj se sada po potrebi može i ručno urediti (grupirati linije i sl.).

5.10.2 Kreiranje zahtjeva za takt

Definirati zahtjeve za takt na način opisan u poglavlju 3.11.3. i pohraniti rezultat u *.xdc* datoteku.

Nakon što je *.xdc* datoteka napravljena, potrebno je ponoviti sintezu.

5.11 Implementacija

Provesti implementaciju na način opisan u poglavlju 3.12.

5.12 Generiranje datoteke za programiranje

Provesti generiranje datoteke za programiranje na način opisan u poglavlju 3.13.

5.13 Programiranje sklopa

Provesti programiranje FPGA sklopa na način opisan u poglavlju 3.14. Provjeriti ispravnost rada sklopa i po potrebi otkloniti greške

6. Vježba 4 - Ugrađeni procesor

6.1 Uvod

Ovo poglavlje opisuje rad s ARM procesorom i pripadajućim periferijama. Ilustrira razvoj sklopovlja i programske podrške. Razvoj sklopovlja izvodi se u programu Vivado, a koriste se jezgre i blokovski dizajn. Programska podrška razvija se u programu Vitis.

U primjeru će osim procesorskog sustava biti korištena i programabilna logika. Radi jednostavnosti primjera, između ovih dvaju sustava neće biti interakcije. Programska podrška bit će razvijena bez korištenja upravljačkih programa.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeće poglavlje iz materijala za predavanja:

6 Računalni sustav u SoC okruženju

6.2 Zadatak

Potrebno je razviti sklopovlje i programsku podršku koji pali i gasi svjetleće diode LD0 i LD9 na razvojnom sustavu ZedBoard. Dioda LD0 potrebno je paliti i gasiti pomoću jezgre *led_blinker* koja je razvijena u vježbi 2, s periodom između 1 s i 2 s. Diodu LD9 potrebno je paliti i gasiti programski, sličnim periodom kao i LD0.

Pristup periferijama procesorskog sustava realizirati izravno upisom u registre, bez pomoći upravljačkih programa.

Jezgra *led_blinker* treba raditi na vanjskom taktu frekvencije 100 MHz. Za njen reset upotrijebiti tipku BTNC.

6.3 Priprema radne okoline

Sve datoteke koje pripadaju ovoj vježbi bit će pohranjene u zasebnu podmapu mape **prezime_ime**. U skladu s tim, za ovu vježbu potrebno je napraviti mapu

prezime_ime\v4

6.4 Pokretanje programa Vivado

Pokrenuti program Vivado. Ako se je pritom automatski učitao neki projekt, zatvoriti ga pomoću komande

File/Close Project

6.5 Kreiranje novog projekta

Na način opisan u poglavlju 3.5 kreirati novi projekt. Projekt je potrebno pohraniti u mapu *prezime_ime\v4*.

6.6 Uključivanje jezgre u dizajn

Na način opisan u poglavlju 5.6 napraviti repozitorij i dodati u njega jezgru *led_blinker* koja je razvijena u vježbi 2. Za repozitorij kreirati mapu *prezime_ime\v4\my_cores*.

6.7 Razvoj sklopovlja

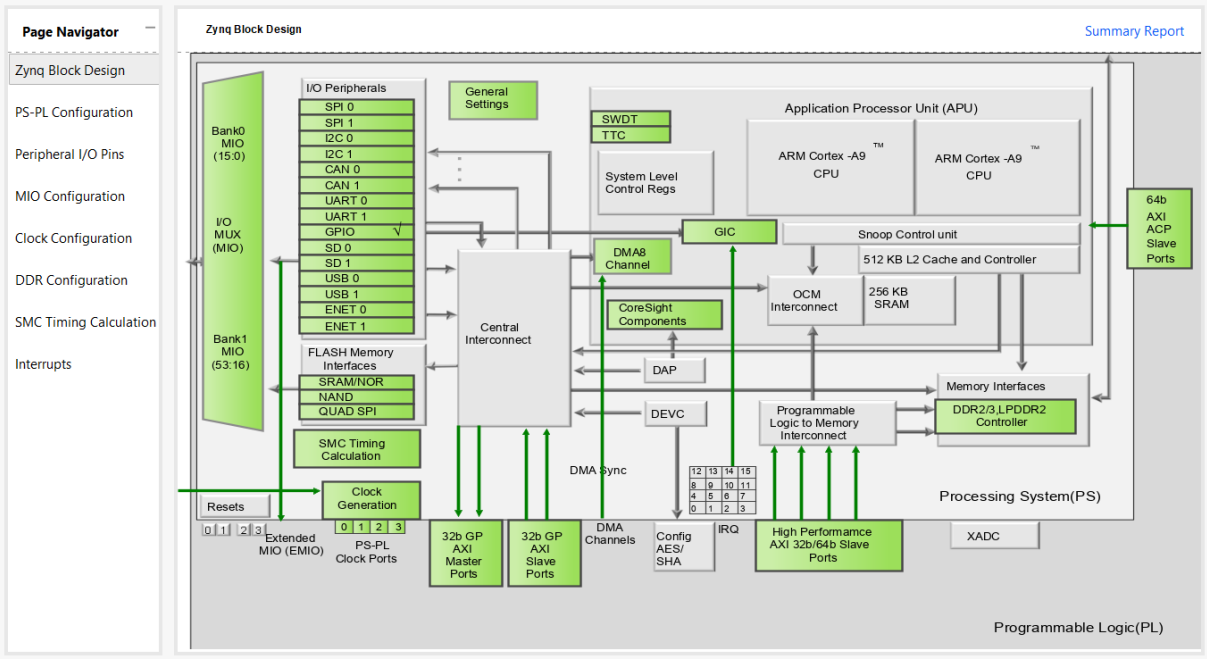
6.7.1 Crtanje sheme

Pokrenuti komandu

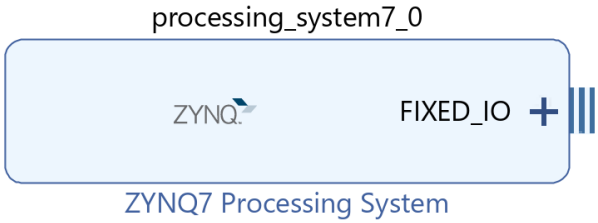
Flow/Create Block Design

Upisati ime koje se želi dati dizajnu (*Design Name*). Preporučuje se odabrati ime *es* (kratica od *Embedded System*) te odabrati *OK*.

U shemu ćemo najprije dodati procesorski sustav. Ovaj sustav izveden je u obliku jezgre. U prozoru *Diagram* pomoću desne tipke miša odabrati *Add IP*, te odabrati jezgru *ZYNQ7 Processing System*. Nakon pojave jezgre na shemi odabrati jezgru (na shemi) te pomoću desne tipke miša odabrati *Customize Bock*, nakon čega će se pojaviti sučelje za konfiguriranje jezgre prikazano slikom 6.1. Redom odabrati sve elemente liste u lijevom prozoru te detaljno proučiti koji dijelovi procesorskog sustava mogu biti konfigurirani. U ovoj vježbi potrebno je **isključiti sve podsustave osim GPIO MIO**. Dobivena shema mora imati oblik prikazan slikom 6.2.



Slika 6.1 Sučelje za konfiguriranje jezgre *ZYNQ7 Processing System*.

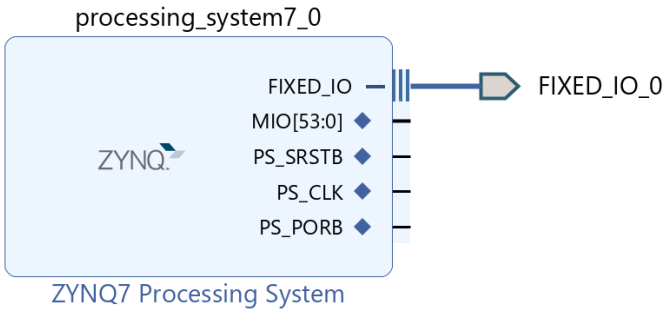


Slika 6.2 Minimalna konfiguracija ugrađenog procesorskog sustava.

Skup priključaka FIXED_IO potrebno je spojiti na vanjske priključke sklopa Zynq. Odabrati ove priključke te pomoću desnog gumba miša odabrati komadu

Make External

Prema potrebi, odabrati priključak i pomoću komande *External Port Properties* promijeniti ime. Odabirom znaka + u grafičkom prikazu jezgre može se dobiti detaljniji uvid u grupe priključaka skupa FIXED_IO, kao što prikazuje slika 6.3. Uočiti da skup FIXED_IO sadrži signale MIO[53:0]. Na razvojnom sustavu ZedBoard, ovi signali spojeni su preko vanjskih priključaka na razne komponente. U ovom primjeru bit će korištena samo jedna od njih – svjetleća dioda LD9 koja je spojena na priključak MIO[7].



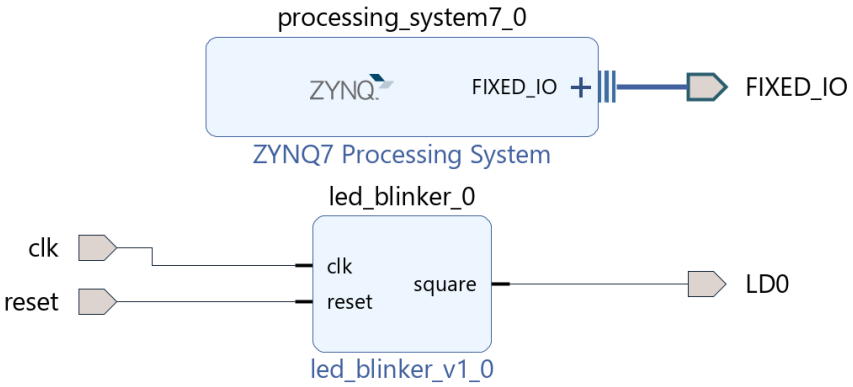
Slika 6.3 Spoj procesorskog sustava s vanjskim priključcima sklopa Zynq.

Sada je na shemu potrebno dodati sklopovlje koje će biti implementirano u programabilnoj logici. U prozoru *Diagram* pomoću desne tipke miša odabrati *Add IP*, te odabrati jezgru *led_blinker*. Nakon pojave jezgre na shemi, odabrati jezgru (na shemi) i pomoću desne tipke miša odabrati *Customize Block*. Podesiti parametar *CLK Divider Width* na odgovarajuću vrijednost. Spojiti *led_blinker* s vanjskim priključcima sklopa Zynq. Spajanje se izvodi pomoću komande *Make External*. Desnim gumbom miša selektirati jedan po jedan vanjski priključak i pomoću komande *External Port Properties* promijeniti imena priključaka da odgovaraju onima u zadatku.

Konačna shema prikazana je slikom 6.4. Prema potrebi, u prozoru *Diagram* pomoću desne tipke miša pokrenuti *Regenerate Layout*.

U prozoru *Diagram* pomoću desne tipke miša pokrenuti komandu *Validate Design* i prema potrebi ispraviti greške.

Gotov dizajn pohraniti pomoću komande *File/Save Block Design*.



Slika 6.4 Shema računalnog sustava.

6.7.2 Generiranje HDL modela pojedinih komponenata

Na način opisan u poglavlju 5.7.2 podesiti VHDL kao ciljani jezik (*Target Language*) te generirati modele pojedinih komponenata.

6.7.3 Generiranje krovnog HDL modela

Na način opisan u poglavlju 5.7.3 generirati krovni model.

6.8 Analiza dizajna

Provesti analizu dizajna na način opisan u poglavlju 3.7. Analizirati dobivenu shemu, provjeriti je li ona u skladu s očekivanjima te po potrebi ispraviti greške.

6.9 Sinteza

Provesti sintezu na način opisan u poglavlju 3.10.

6.10 Definiranje zahtjeva na implementaciju

6.10.1 Definiranje priključaka

Definiranje zahtjeva na implementaciju opisano je u poglavlju 3.11. Ovdje je potrebno kreirati datoteku *es.xdc* (vidi poglavlje 3.11.1) i definirati priključke (vidi poglavlje 3.11.2).

Uočiti da je u prozoru *I/O Ports* na vanjske priključke već spojen cijeli skup signala FIXED_IO, pa je tako spojen i MIO[7] koji pogoni diodu LD9. Ove priključke spojio je sam alat. Ostale signale sa sheme potrebno je "ručno" spojiti s priključcima sklopa Zynq. U ovom primjeru radi se o još tri signala. Dva signal su ulazna,

- clk
- reset

a jedan izlazni,

- LD0

Spojiti ove signale na način dan u zadatku te podesiti parametre priključaka

- priključak
- IO Standard
- Drive Strength
- Pull Type
- Off-Chip Termination

Za odabir ovih parametara poslužiti se podacima danim na predavanju ili priručnikom za rad s razvojnim sustavom ZedBoard.

Nakon što su priključcima dodijeljeni parametri, ne zaboraviti parametre pohraniti u datoteku *es.xdc* pomoću komande

File/Constraints/Save

Nakon definiranja priključaka otvoriti datoteku *es.xdc* i provjeriti njen sadržaj.

6.10.2 Kreiranje zahtjeva za takt

Definirati zahtjeve za takt na način opisan u poglavlju 3.11.3. i pohraniti rezultat u *.xdc* datoteku.

Nakon što je *.xdc* datoteka napravljena, potrebno je ponoviti sintezu.

6.11 Implementacija

Provesti implementaciju na način opisan u poglavlju 3.12.

6.12 Generiranje datoteke za programiranje

Provesti generiranje datoteke za programiranje na način opisan u poglavlju 3.13.

6.13 Programiranje sklopa

Ako bi sada konfiguracija bila učitana u sklop Zynq, radila bi samo dioda LD0. Za ispravan rad diode LD9 nedostaje programska podrška procesorskog sustava. Postupak razvoja ove programske podrške opisan je u daljnjem tekstu.

Dioda koja se pali i gasi nezavisno od preostalih sklopova, kao što je LD0 u ovom slučaju, često se dodaje u programabilnu logiku zato da bi se "na oku" držala ispravnost cijelog razvojnog lanca (osobno računalo, Vivado i Vitis, razvojni sustav, kabeli itd.).

Ako u ovoj fazi razvoja ugradbenog sustava postoji potreba za programiranjem FPGA sklopa, to se može napraviti na način opisan u poglavlju 3.14.

6.14 Prijenos konfiguracije sklopovlja u programsko okruženje

Nakon što je završen razvoj sklopovlja te napravljena njegova implementacija, potrebno je prenijeti konfiguraciju sklopovlja u programsko okruženje. Konfiguracija se prenosi pomoću komande

File/Export/Export Hardware

Uključiti opciju *Include bitstream*. Dodijeliti ime XSA datoteci i podesiti put na željenu mapu (obično je već podešen). Za XSA datoteku preporučuje se koristiti ime *zynq1.xsa*. Dovršiti postupak pomoću *Next* i *Finish*. Rezultat će biti pohranjen u XSA datoteku.

Napomena:

Prije pokretanja prijenosa sklopovlja u programsko okruženje potrebno je zatvoriti *Elaborated Design* i otvoriti *Block Design*.

6.15 Programska podrška temeljena na izravnom pristupu sklopovlju

6.15.1 Pokretanje programa Vitis

Vitis se može pokrenuti iz Windows okruženja kao

Start/Xilinx Design Tools/Xilinx Vitis 2020.2

ili izravno iz Vivada pomoću

Tools/Launch Vitis IDE

Nakon pokretanja Vitisa, potrebno je provjeriti je li program u ispravnoj radnoj mapi i po potrebi ju promijeniti. U ovom primjeru radna mapa je *prezime_ime\v4*. Radna mapa mijenja se pomoću komande:

File/Switch Workspace

Napomena:

Da se izbjegne moguća pogreška, kod prvog poziva Vitisa u tekućem projektu preporučljivo je ovaj alat pokrenuti ručno (a ne iz Vivada) te na početku rada promijeniti radnu mapu. Kasnije će program automatski startati iz mape projekta.

6.15.2 Kreiranje platforme

Pokrenuti komandu

File/New/Platform Project

te upisati ime platforme. U ovom primjeru nazvat ćemo ju *zynq1*. Potrebno je kreirati novu platformu iz opisa sklopovlja (XSA). To se izvodi odabirom .xsa datoteke, u ovom slučaju *prezime:ime\v4\zynq1.xsa*. Odabrati procesor *ps7_cortexa9_0* te operacijski sustav *standalone*. Uočiti da je odabirom procesora i operacijskog sustava zapravo definirana domena. Odabrati *Finish*.

6.15.3 Dodavanje nove domene

Ako je potrebno definirati još jednu ili više domena, potrebno je otvoriti datoteku koja sadrži konfiguraciju platforme, *platform.spr*, u njoj odabrati ime platforme te desnim gumbom miša odabrati komandu *Add Domain.*

U ovom primjeru ne treba dodavati novu domenu.

6.15.4 Kreiranje prazne aplikacije

Aplikacija (*software*) bit će vezana uz platformu. Pokrenuti komandu

File/New/Application Project

te odabrati platformu uz koju je vezana aplikacija (u ovom primjeru to je platforma *zynq1*). Odabrati *Next*.

Odabrati *System Project* u kojem će biti otvoren nova aplikacija tj. njen *Application Project*. S obzirom da još nije napravljen niti jedan *System Project*, potrebno je odabrati *Create new ...* . U polje *Application project name* upisati ime aplikacije (u ovom primjeru nazvat ćemo ju *sw1*). Uočiti da je ime za *System Project* ponuđeno u obliku *sw1_system*, a može se i promijeniti ako za to postoji potreba. U ovom primjeru zadržati ponuđeno ime. Odabrati *Next*.

Odabrati domenu kojoj će biti pridružena aplikacija *sw1*. Odabrati *Next*. Odabrati predložak (*template*) koji predstavlja praznu aplikaciju (*Empty Application*) i na kraju odabrati *Finish*.

6.15.5 Razvoj aplikacije

U prozoru *Explorer* odabrati *sw1_system/sw1/src*, te pritiskom da desni gumb miša odabrati komandu

New/File

Upisati ime datoteke koja će sadržavati C kod. Paziti da u imenu postoji nastavak '.c'. Nazvati datoteku *appl.c*, te odabrati *Finish*.

Napisati C kod aplikacije. Prema potrebi, poslužiti se primjerom s predavanja. Pohraniti datoteku pomoću komande

File/Save

ili

File/Save All

6.15.6 Opcije za povezivanje i punjenje

Opcije za povezivanje i punjenje (*linker/loader*) sadržane su u datoteci *lscript.ld* koja se nalazi u *src* direktoriju tekućeg aplikacijskog projekta (u ovom slučaju *sw1_system/sw1/src*). Selektirati ovu datoteku te pomoći desnog gumba miša pokrenuti komandu *Open*. Sadržaj datoteke moguće je editirati kroz izvorni kod (*Source*) ili kroz pripadajući editor (*Summary*).

Odabrati *Summary*. Uočiti da su na raspolaganju samo dva memorijska prostora te da oni predstavljaju dvije banke unutrašnje memorije (*On Chip Memory*, OCM). U ovom primjeru nije potrebno mijenjati memorijsko mapiranje.

6.15.7 Prevođenje programske podrške

Postupak prevođenja programske podrške (*build*) pokreće se odabirom projekta (*Platform*, *System* ili *Application Project*) te pokretanjem komande *Build Project* koja se dobiva pritiskom desnog gumba miša.

Napomene:

Prvo je potrebno prevesti *Platform Project* pa tek onda *Application Project*.

Komanda iz izbornika

Project/Clean

briše rezultate prevođenja odabranog projekta. Osim toga, moguće je uključiti opciju *Clean all projects*. Komanda iz izbornika

Project/Build All

prevodi sve projekte. Nakon pokretanja ove komande, paziti na redoslijed prevođenja.

Ako se tijekom prevođenja pojavi greška koja ukazuje da ne postoji datoteka *xparameters.h*, radi se o pogrešnom redoslijedu prevođenja. Za detaljno objašnjenje pogledati bilješku u poglavlju 9.11.

6.16 Pokretanje i uhodavanje programske podrške

Vitis podržava dva alata za uhodavanje (*debugger*)

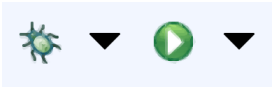
- *Xilinx System Debugger*
- *GNU Debugger (GDB)*

Prvi alat je dio Vitis programskog okruženja dok je drugi alat vanjski, a poziv se iz Vitis sučelja. U ovom primjeru bit će korišten *Xilinx System Debugger*.

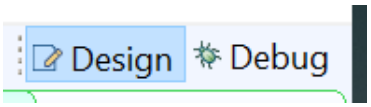
Programska podrška koju uhodavamo izvršava se na stvarnom sklopovlju, u ovom slučaju na sustavu ZedBoard. Prije pokretanja programske podrške, u ZedBoard se učitava konfiguracija sklopovlja i izvršna inačica programske podrške. Izvršavanje programske podrške moguće je pokrenuti na jedan od dva načina:

- *Run*
- *Debug*

U *Run* načinu rada programska podrška se izvršava bez zaustavljanja. *Debug* način koristi se za uhodavanja programske podrške (*debugging*). Programska podrška se pokreće, ali će njeno izvršavanje biti zaustavljeno na unaprijed definiranim točkama prekida, npr. na ulazu u funkciju *main()*. Komande za pokretanje programske podrška na oba načina nalaze se u alatnoj traci, a imaju oblik



Tijekom razvoja programske podrške Vivado prikazuje prozore razvojnog okruženja u pogledu *Design*. S početkom uhodavanja Vivado automatski mijenja pogled na *Debug*. Promjena pogleda može se izvesti i ručno odabirom jedne od sljedećih komandi u alatnoj traci



Za pokretanje ili uhodavanje programske podrške na sklopovlju potrebno je u prozoru *Explorer* odabrati željeni sistemski projekt, u ovom slučaju *sw1*. Nakon toga je u alatnoj traci u padajućem izborniku u dijelu



potrebno odabrati komandu

Run As/Launch Hardware

ili

Debug As/Launch Hardware

U oba slučaja, na sustav Zedboard bit će prenesena konfiguracija koja obuhvaća sklopovlje i programsku podršku. U prvom slučaju program će biti pokrenut na sklopovlju te će biti moguće provjeriti ispravnost rada, u ovom slučaju paljenja i gašenja svjetlećih dioda. U drugom slučaju bit će pokrenuto uhodavanje, pri čemu će program (koji se i ovdje izvršava na sklopovlju) biti zaustavljen. Program je sada moguće pokretati liniju po liniju, dodavati točaka prekida, analizirati sadržaje registara i memorije i sl.

Važna napomena:

Prije pokretanja cijele programske podrške na sklopovlju kao prije započinjanja uhodavanja potrebno je odabrati mišem sistemski projekt (mora biti obojan plavom bojom).

7. Vježba 5 - Upravljački programi

7.1 Uvod

Ovo poglavlje razmatra nekoliko aspekata dizajna računalnih sustava temeljenih na sklopu Zynq. Prvo, bit će ilustriran razvoj programske podrške temeljen na upravljačkim programima, pri čemu će biti korištene dvije periferije – GPIO i UART. Drugo, razvijena programska podrška bit će smještena u vanjsku dinamičku memoriju. Treće, bit će pokazano kako se u programskom okruženju Vivado/Vitis mijenja sklopovlje i ažurira pripadajuća programska podrška. Također, bit će pokazano što treba učiniti kad se kopira radna mapa cijelog projekta.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeće poglavlje iz materijala za predavanja:

7 Standalone OS

7.2 Zadatak

U prethodnoj vježbi razvijeno je sklopovlje i programska podrška koja pali i gasi dvije svjetleće diode. Jednu diodu pogoni programabilna logika, a drugu programska podrška koja ne koristi upravljačke programe.

Sklopovlje iz prethodne vježbe bit će dijelom iskorišteno u ovoj vježbi. U njega je potrebno dodati vanjsku dinamičku memoriju (DDR) te sklop za serijsku komunikaciju (UART).

Staru programsku podršku potrebno je izmijeniti. Paljenje i gašenje svjetleće diode LD9 potrebno je izvesti korištenjem upravljačkog programa **GpioPS**. Nadalje, potrebno je dodati programsku podršku koja preko serijskog sučelja ispisuje poruku "Vježba 5 je napravljena." Pritom je potrebno koristiti upravljački program **UartPs**.

7.3 Priprema radne okoline

U ovoj vježbi bit će iskorišteno sklopovlje i dio programske podrške iz prethodne vježbe. Da bi prethodna vježba ostala sačuvana u cijelosti, potrebno je kopirati mapu

prezime_ime\v4

u novu mapu koju treba nazvati

prezime_ime\v5

Paziti da u daljnjem radu programi Vivado i Vitis budu otvarani u mapi *prezime_ime\v5*.

7.4 Pokretanje programa Vivado

Pokrenuti program Vivado. Ako se pritom automatski učitao neki projekt, zatvoriti ga pomoću komande

File/Close Project

S obzirom da je radna mapa projekta kopirana s drugog mjesta na disku, preporučljivo je resetirati cijeli projekt. S obzirom da Vivado u izborniku nema naredbu za resetiranje projekta, resetiranje se pokreće iz TCL konzole pokretanjem komande

reset_project

7.5 Razvoj sklopovlja

7.5.1 Izmjena sheme

Pokrenuti komandu

Flow/Open Block Design

nakon čega će biti otvorena shema koja pripada dizajnu *es*. Na shemu je potrebno dodati vanjsku podatkovnu memoriju i serijsko sučelje.

7.5.1.1 Spajanje vanjske memorije

Pomoću miša odabrati jezgru *ZYNQ7 Processing System*, te pomoću desne tipke odabrati *Customize Bock*, nakon čega će se pojaviti sučelje za konfiguriranje jezgre (slika 6.1). U lijevom prozoru odabrati skup parametara *DDR Configuration* te uključiti opciju *Enable DDR*. Ova opcija uključuje DDR kontroler. Memorijski kontroler je kompatibilan s velikim brojem memorija koje se mogu naći na tržištu. Međutim, potrebno ga je konfigurirati, što uključuje širine sabirnica i vremenske odnose pojedinih signala. Specifikacija vremenskih odnosa ovisi o samoj memoriji tj. memorijskom sklopu (integriranom krugu) te o topologiji tiskane pločice (duljinama vodova, kašnjenjima i sl.).

Na ZedBoardu se nalaze dva memorijska sklopa **MT41J128M16 HA-15E**, koji čine vanjsku dinamičku memoriju kapaciteta 512 MB. Za njihov ispravan rad potrebno je podesiti sljedeće parametre memorijskog kontrolera:

- DDR Controller Configuration
  - Memory Type DDR3
  - Memory Part **MT41J128M16 HA-15E**
  - Effective DRAM Bus Width 32 Bit
  - ECC Disabled
  - Burst Length 8
  - DDR 533.333333
  - Internal Vref Enabled
  - Junction Temperature Normal
- Memory Part Configuration
  - ovi parametri automatski se učitavaju ne temelju odabranog memorijskog sklopa
- Training/Board Details User Input
  - DRAM Training
    - Write leveling Enabled
    - Read gate Enabled
    - Read data eye Enabled

- DQS to Clock Delay (ns)
  - DQS0 0.025
  - DQS1 0.028
  - DQS2 0.001
  - DQS3 0.001
- Board Delay (ns)
  - DQ[7:0] 0.41
  - DQ[15:8] 0.411
  - DQ[23:16] 0.341
  - DQ[31:24] 0.358
- Advance Options Disabled

Proučiti navedene parametre, uočiti njihovu ulogu te unijeti parametre u odgovarajuća polja za konfiguraciju memorijskog kontrolera. Odabrati *OK*.

Uočiti da se nakon uključivanja memorijskog kontrolera na procesorskom sustavu pojavio skup priključaka DDR. Njega je sada potrebno spojiti na vanjske priključke sklopa Zynq. Odabrati ove priključke te pomoću desnog gumba miša odabrati komadu

Make External

Napomena

Ako bi na laboratorijskoj vježbi bila na raspolaganju verzija ZedBoarda s drugačijim DDR memorijskim sklopom od one čija je konfiguracija dana u gornjem tekstu, parametri za konfiguriranje bit će dani na samoj vježbi.

7.5.1.2 Spajanje serijskog sučelja

Pomoću miša odabrati jezgru *ZYNQ7 Processing System*, te pomoću desne tipke odabrati *Customize Bock*. U lijevom prozoru sučelja za konfiguriranje jezgre odabrati skup parametara *Peripheral I/O Pins* te uključiti periferiju UART1. Ne uključivati opciju *Modem signals*.

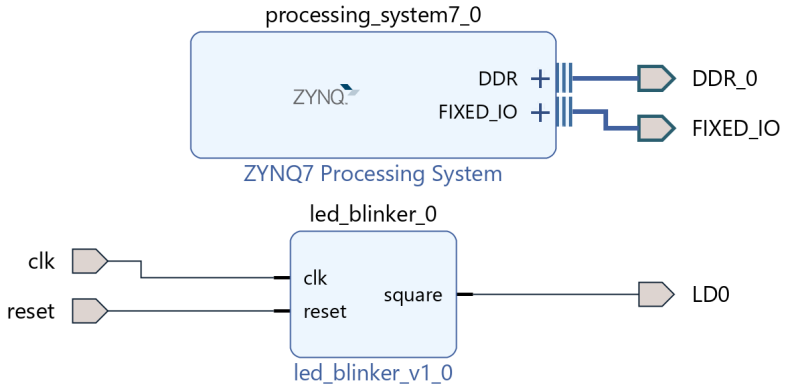
U desnom prozoru, koji sadrži spoj signala periferija s vanjskim priključcima sklopa Zynq, odabrati spoj periferije UART1 s priključcima MIO[49:48] (spoj je označen zelenom bojom). Uočiti da su ovi priključci sada odspojeni od sklopa GPIO.

U lijevom prozoru sučelja za konfiguriranje jezgre odabrati skup parametara *MIO Configuration* te u dijelu *MIO Configuration* podesiti *Bank 0 I/O Voltage* na LVCMOS 3.3V, a *Bank 1 I/O Voltage* na LVCMOS 1.8V. U donjem dijelu prozora provjeriti rezultat ovog podešavanja. Odabrati *OK*.

Konačna shema prikazana je slikom 7.1. Prema potrebi, u prozoru *Diagram* pomoću desne tipke miša pokrenuti *Regenerate Layout*.

U prozoru *Diagram* pomoću desne tipke miša pokrenuti komandu *Validate Design* i prema potrebi ispraviti greške.

Gotov dizajn pohraniti pomoću komande *File/Save Block Design*.



Slika 7.1 Shema računalnog sustava.

7.5.2 Generiranje HDL modela pojedinih komponenata

Na način opisan u poglavlju 5.7.2 podesiti VHDL kao ciljani jezik (*Target Language*) te generirati modele pojedinih komponenata.

7.5.3 Generiranje krovnog HDL modela

Na način opisan u poglavlju 5.7.3 generirati krovni model.

7.6 Analiza dizajna

Provesti analizu dizajna na način opisan u poglavlju 3.7. Analizirati dobivenu shemu, provjeriti je li ona u skladu s očekivanjima te po potrebi ispraviti greške.

7.7 Sinteza

Provesti sintezu na način opisan u poglavlju 3.10.

7.8 Definiranje zahtjeva na implementaciju

Datoteka koja sadrži zahtjeve za implementaciju već je kreirana, a zove se *es.xdc*. Njeno kreiranje provedeno je tijekom prethodne vježbe, a način kreiranja opisan je u poglavlju 3.11.1.

Sada je potrebno dodati nove priključke na način opisan u poglavlju 3.11.2. Pokrenuti komande

Flow/Open Synthesized Design
Window/ I/O Ports

Proučiti dobiven prozor *I/O Ports*. Uočiti da je na vanjske priključke već spojen cijeli skup signala FIXED_IO, pa je tako spojen i MIO[7] koji pogoni diodu LD9. Nadalje, već od prethodne vježbe spojeni su signali clk, reset i LD0 koji se koriste u programabilnoj logici. Signali koji su dodani u ovoj vježbi također su vidljivi. To su signali koje koristi DDR memorija. Uočiti da su ovi signali automatski spojeni na vanjske priključke te da su već podešeni njihovi parametri.

Nakon što je provjerena definicija priključaka (i u slučaju potrebe popravljene greške), otvoriti datoteku *es.xdc* i provjeriti njen sadržaj. Uočiti da su u datoteci već od prethodne vježbe definirani zahtjevi za takt. (Ako zbog bilo kojeg razloga nedostaje redak s definicijom takta, potrebno je definirati zahtjeve za takt na način opisan u poglavlju 3.11.3. te pohraniti rezultat u *.xdc* datoteku.)

Ako je u *.xdc* datoteci bilo što mijenjano, potrebno je ponoviti sintezu.

7.9 Implementacija

Provesti implementaciju na način opisan u poglavlju 3.12.

7.10 Generiranje datoteke za programiranje

Provesti generiranje datoteke za programiranje na način opisan u poglavlju 3.13.

7.11 Prijenos konfiguracije sklopovlja u programsko okruženje

Na način opisna u poglavlju 6.14 napraviti novu XSA datoteku. Dodijeliti joj isto ime kao i u prethodnoj vježbi (presnimiti preko stare datoteke). Kod odabira imena provjeriti je li u polju *Export to* ispravno postavljen put mape pete vježbe, *prezime_ime\v5*.

7.12 Programska podrška temeljena na upravljačkim programima

7.12.1 Pokretanje programa Vitis

Pokrenuti Vitis prema uputama u poglavlju 6.15.1. Paziti da on bude pokrenut u ispravnoj radnoj mapi i po potrebi ju promijeniti. U ovom primjeru radna mapa je *prezime_ime\v5*.

7.12.2 Ažuriranje platforme

S obzirom da smo prethodnim poglavljima promijenili shemu, platforma (*Platform Project*) više nije ažurna. Da bi se generiralo okruženje koje odgovara novom sklopovlju, potrebno je provesti radnje opisane u daljnjem tekstu.

Odabrati *Platform Project* (u ovom slučaju *zynq1*) i pomoću desnog gumba miša pokrenuti komandu *Clean Project*. Odabrati *Platform Project* i pomoću desnog gumba miša pokrenuti komandu *Update Hardware Specification*. Paziti da bude odabrana ispravna XSA datoteka. Odabrati *OK*.

Desnim gumbom miša odabrati datoteku s konfiguracijom platforme, *platform.spr*, te ju otvoriti pomoću komande *Open*. U domeni *standalone on ps7_cortexa9_0* odabrati *Board Support Package*. Uočiti da se u desnom prozoru pojavio skup upravljačkih programa koji su vezani uz spomenutu platformu. Proučiti ih i uočiti program *uartps* koji će biti korišten za rad sa serijskom vezom odnosno sklopom UART1. Pokrenuti komandu *Modify BSP Settings ....* U lijevom dijelu prozora odabrati *standalone* te proučiti dobivenu konfiguraciju operacijskog sustava. S obzirom da na prethodnoj vježbi UART1 nije bio uključen, *stdin* i *stdout* preusmjereni su na *coresight* podsustav za uhodavanje. Sada ih je potrebno preusmjeriti na UART1. To se izvodi odabirom *ps_uart_1* u polju *Value*.

Odabrati *Platform Project* i pomoću desnog gumba miša pokrenuti komandu *Build Project*.

7.12.3 Ažuriranje aplikacijskog i sistemskog projekta

Nakon što je ažurirana sklopovska platforma, potrebno je ažurirati i postojeći aplikacijski te sistemski projekt (*sw1* odnosno *sw1_system*).

Odabrati sistemski projekt (*System Project*) te u njemu pronaći i otvoriti datoteku *sw1_system.sprj*, koja sadrži postavke sistemskog projekta (*System Project Settings*). U prozoru s postavkama, na kartici *General*, pored naziva *Platform*, odabrati gumb '**...**'. U prozoru za promjenu platforme (*Change Platform*) odabrati komandu *Manage* i podesiti platformu iz željene (trenutne) radne mape. Da se izbjegnu greške na laboratorijskim vježbama, preporučuje se ukloniti s popisa nepotrebne platforme. Odabrati *OK* i ponovo *OK*.

Generirati novu skriptu za konfiguraciju linkera (*Linker Script*). Odabrati aplikaciju (*Application Project*) u kojoj se generira nova skripta. U ovom slučaju radi se o aplikaciji *sw1*. Pomoću desnog gumba na mišu odabrati komandu *Generate Linker Script*.

Sada je moguće izgraditi programsku podršku odabirom sistemskog projekta (*System Project*) i pokretanjem komandu *Build Project* pomoću desnog gumba miša. Međutim, programska podrška će u ovoj vježbi biti modificirana na način opisan u daljnjim poglavljima.

Napomena:

U ovom primjeru mogli smo krenuti i od praznog Vitis projekta te kreirati platformski, sistemski i aplikacijski projekt na način opisan u poglavljima 6.15.2 i 6.15.4. U tom slučaju platforma bi bila automatski konfigurirana. S obzirom da se radi o jednostavnom projektu, to bi bilo čak i brže nego postupak koji smo upravo proveli. Međutim, u složenim projektima, opisan način modifikacije platforme i aplikacije je mnogo kraći od ponovne izrade pojedinih dijelova projekta.

7.12.4 Izrada aplikacije

Aplikacija koju je potrebno izraditi u ovom primjeru mora paliti i gasiti svjetleći diode LD9 te ispisati poruku "Vježba 5 je napravljena.". Pritom će biti korišteni upravljački programi **GpioPS**. i **UartPs**. Način korištenja ovih upravljačkih programa dan je u materijalima za predavanja. Također, dokumentacija upravljačkih programa može se pronaći u okviru platforme.

U platformskom projektu (u našem slučaju radi se o projektu *zynq1*) ponovo otvoriti datoteku *platform.spr*. U domeni *standalone ...* odabrati *Board Support Package*. Odabrati u desnom prozoru dokumentaciju upravljačkog programa *ps_7_uart_1* na koju pokazuje *Documentation Link* i proučiti što ona sadrži.

U prozoru *Explorer* odabrati *sw1_system/sw1/src*, te otvoriti datoteku *appl.c*, koja sadrži kod s prethodne vježbe. Napisati novi C kod aplikacije. Prema potrebi, poslužiti se primjerima s predavanja. Na početku programske podrške potrebno je inicijalizirati upravljačke programe **GpioPS**. i **UartPs**. Nakon toga treba inicijalizirati sklopove GPIO i UART. Nakon inicijalizacije ispisati zadanu poruku preko serijskog sučelja, a na kraju u beskonačnoj petlji paliti i gasiti svjetleću diodu.

Pohraniti datoteku pomoću komande

File/Save

ili

File/Save All

7.12.5 Opcije za povezivanje i punjenje

Opcije programa za povezivanje i punjenje (*linker/loader*) sadržane su u datoteci *lscript.ld* koja se nalazi u *src* direktoriju tekućeg aplikacijskog projekta (u ovom slučaju *sw1_system/sw1/src*). Odabrati ovu datoteku te pomoći desnog gumba miša pokrenuti komandu *Open*. Poručiti konfiguraciju (*Summary*). Uočiti da je, za razliku od prethodne vježbe, osim OCM memorije, na raspolaganju i DDR memorija (*ps7_ddr_0*).

U dijelu koji određuje mapiranje memorije, svi programski odsječci trebali bi biti mapirani u DDR memoriju. Nadalje, veličina stoga i *heap*-a ne bi trebala biti manja od 0x2000 bajtova.

7.12.6 Prevođenje programske podrške

Prevesti programsku podršku prema uputama u poglavlju 6.15.7.

7.13 Uhodavanje na sustavu ZedBoard

7.13.1 Svjetleće diode

Učitati konfiguraciju koja sadrži sklopovlje i programsku podršku u sustav ZedBoard prema uputama u poglavlju 6.16. Nakon pokretanja trebale bi se paliti i gasiti diode L0 i L9. Ako diode ne rade, pronaći grešku i ispraviti ju.

7.13.2 Serijska veza

Za primanje poruka koje procesorski sustav šalje preko serijske veze bit će korišteno osobno računalo na kojem će biti pokrenut program za emulaciju terminala pod nazivom **PuTTY**.

Pomoću USB kabela povezati osobno računalo i USB sučelje ZedBoarda koje sadrži UART/USB adapter. Na ZedBoardu to jedan od dvaju USB konektor koji se nalaze jadan do drugog, bliži sklopki za uključivanje napajanja.

U Windows okruženju otvoriti **Device Manager** pomoću

Control Panel/System/Device Manager/Ports (COM & LPT)

te pronaći komponentu koja predstavlja UART/USB adapter. Komponenta ima naziv

USB Serial Device (COMx)

ili slično tome, ovisno o verziji adaptera. Pronaći na koje sučelje je adapter spojen. Sučelje je nazvano **COMx** gdje je **x** broj.

Pokrenuti program **PuTTY**. Konfigurirati parametre prijenos u skladu s parametrima sklopa UART1:

Session

Serial line:	COMx	(x predstavlja broj COM sučelja)
Speed:	9600 bit/s	
Type:	Serial	

Connection
Serial Line
Serial line to connect: COM8
Configuration
Speed (baud): 9600
Data bits: 8
Stop bits: 1
Parity: None
Flow control: None

Pokrenuti program na ZedBoardu i provjeriti ispis u prozoru koji odgovara konzoli. Prema potrebi uhodati program.

7.14 Završne napomene

Opis radnji koji je potrebno provesti prilikom promjene sklopovlja računalnog sustava sažeto je opisan u poglavlju "9.6 Promjena sklopovlja računalnog sustava".

Opis radnji koji je potrebno provesti prilikom premještanja ili kopiranja radne mape projekta sažeto je opisan u poglavlju "9.7 Premještanje radne mape projekta".

8. Vježba 6 - AXI periferije

8.1 Uvod

Veza između procesorskog sustava i programabilne logike može biti ostvarena preko EMIO i preko AXI sučelja. Ovo poglavlje ilustrira upotrebu AXI sučelja. U njemu će u sustav razvijen tijekom prethodne vježbe biti dodane svjetleće diode i tipke kojima se pristupa preko AXI periferija.

Za uspješno savladavanje vježbe potrebno je proučiti sljedeće poglavlje iz materijala za predavanja:

8 Povezivanje procesorskog sustava i programabilne logike

8.2 Zadatak

U prethodnoj vježbi razvijen je računalni sustav koji ispisuje poruku preko serijskog sučelja te koji pali i gasi svjetleću diodu LD9, spojenu na GPIO sklop preko MIO sučelja. Nadalje, sustav ispisuje poruku preko serijskog sučelja UART1. Sklopovlje sadrži i *led_blinker* koji je implementiran u programabilnoj logici, a pogoni diodu LD0.

Sklopovlje iz prethodne vježbe bit će iskorišteno u ovoj vježbi. U njega je potrebno dodati dvije AXI periferije. Jedna od njih treba biti spojena na svjetleće diode LD1, LD2, LD3, LD4, LD5, LD6 i LD7. Druga treba biti spojena na tipke BTNR i BTNL.

Modificirati programsku podršku na sljedeći način. Odsječak koji neprekidno pali i gasi diodu LD9 modificirati tako da se dioda upali i ugasi 10 puta. Zatim dodati programski odsječak koji u beskonačnoj petlji neprekidno redom pali svjetleće diode udesno ili ulijevo (trčeće svjetlo). Redoslijed paljenja određuje se pritiskom na tipke BTNR i BTNL. Pritisak tipke BTNR mijenja slijed paljenja udesno, a tipke BTNL ulijevo.

U razvoju programske podrške koristiti upravljačke programe.

8.3 Priprema radne okoline

U ovoj vježbi bit će iskorišteno sklopovlje i dio programske podrške iz prethodne vježbe. Da bi prethodna vježba ostala sačuvana u cijelosti, potrebno je kopirati mapu

prezime_ime\v5

u novu mapu koju treba nazvati

prezime_ime\v6

Paziti da u daljnjem radu programi Vivado i Vitis budu otvarani u mapi *prezime_ime\v6*.

8.4 Pokretanje programa Vivado

Pokrenuti program Vivado. Ako se pritom automatski učitao neki projekt, zatvoriti ga pomoću komande

File/Close Project

S obzirom da je radna mapa projekta kopirana s drugog mjesta na disku, preporučljivo je resetirati cijeli projekt. S obzirom da Vivado u izborniku nema naredbu za resetiranje projekta, resetiranje se pokreće iz TCL konzole pokretanjem komande

reset_project

8.5 Razvoj sklopovlja

8.5.1 Izmjena sheme

Pokrenuti komandu

Flow/Open Block Design

nakon čega će biti otvorena shema koja pripada dizajnu *es*. Shemu je potrebno promijeniti prema uputama u daljnjem tekstu.

8.5.1.1 Konfiguriranje procesorskog sustava

Pomoću miša odabrati jezgru *ZYNQ7 Processing System*, te pomoću desne tipke odabrati *Customize Bock*. U lijevom prozoru odabrati skup parametara *PS-PL Configuration* te u desnom prozoru pronaći i omogućiti reset signal opće namjene *FCLK_RESET0_N*. Nadalje, pronaći i omogućiti nadređeno AXI sučelje opće namjene, *M AXI GP0 interface*. Ne uključivati opciju za statičko remapiranje. U lijevom prozoru odabrati skup parametara *Clock Configuration* te u desnom prozoru pronaći i omogućiti PL takt *FCLK_CLK0*.

Odabrati *OK*.

8.5.1.2 Dodavanje AXI prospoja

U prozoru *Diagram* pomoću desne tipke miša odabrati *Add IP* te dodati jezgru *Processor System Reset*. Ponovo odabrati *Add IP* te dodati jezgru *AXI Interconnect*. Odabrati ovu jezgru te pomoću desne tipke miša pokrenuti komandu *Customize Bock*. U izbornicima *Number of Slave Interfaces* i *Number of Master Interfaces* odabrati jedno podređeno i dva nadređena sučelja. Odabrati *OK*.

Ponovo odabrati *Add IP* te dodati jezgru *AXI GPIO*. Ova jezgra pogonit će 7 svjetlećih dioda. Pokrenuti komandu *Customize Bock*. Na kartici *IP Configuration* podesiti *GPIO width* na odgovarajuću širinu te odabrati opciju *All Outputs*. (Ako bi ove opcije slučajno bila nedostupne, odabrati karticu *Board* i pokrenuti komandu *Clear Board Parameters*.)

Preporučljivo je u ovom trenutku spojiti učitane komponente. To je najlakše učiniti korištenjem automatskog spajanja. U prozoru *Diagram* pomoću desnog gumba miša odabrati komandu *Run Connection Automation*. Odabrati sva sučelja čije spajanje želimo prepustiti sustavu. Odabrati *OK*.

Pomoću komande *Add IP* dodati još jednu jezgru *AXI GPIO*. Ova jezgra bit će spojena na dvije tipke. Pomoću komande *Customize Bock* podesiti *GPIO width* na odgovarajuću širinu te odabrati opciju *All Inputs*.

Ponovo pokrenuti komandu *Run Connection Automation*, odabrati sva sučelja čije spajanje želimo prepustiti sustavu te odabrati *OK*.

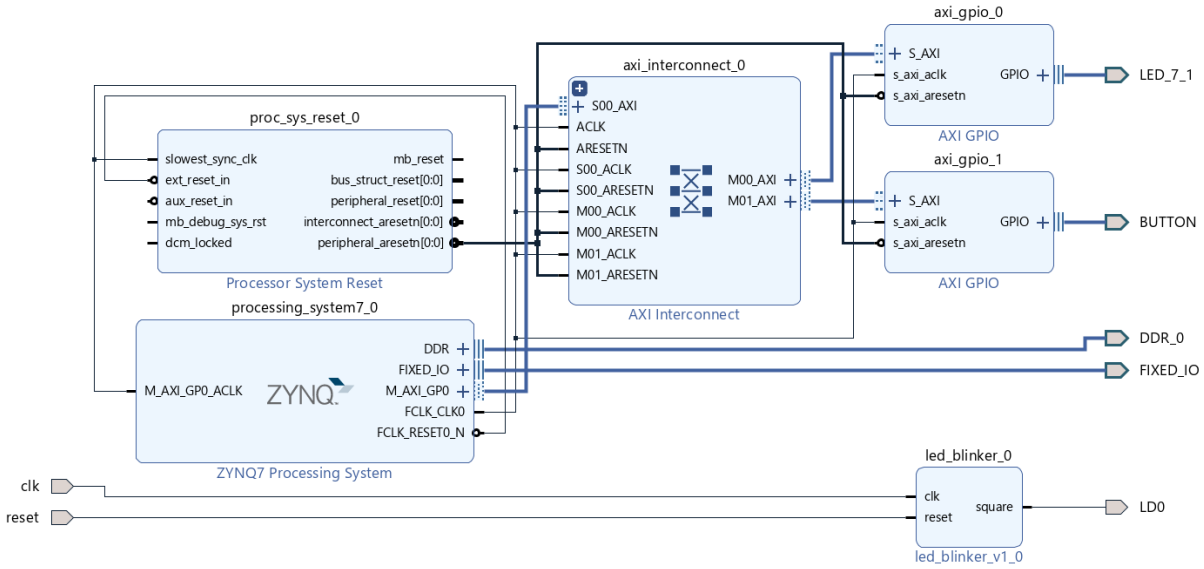
Provjeriti je li nakon automatskog spajanja ostao ispravan broj i polaritet priključaka na komponentama AXI GPIO. Paziti da diode budu spojene na AXI sučelje 0 a tipke na AXI

sučelje 1 na AXI prospoju. Prema potrebi ponoviti komandu *Customize Bock*, pokrenuti komandu *Clear Board Parameters* te ponovo podesiti broj i tip priključaka.

Redom odabrati konektore na koje su spojene komponente *AXI GPIO* te pomoću komande *External Interface Properties* promijeniti nazive konektora u *LED_7_1* odnosno *BUTTON*.

Prema potrebi urediti shemu pomoću komande *Regenerate Layout*. Konačna shema trebala bi izgledati kao ona na slici 8.1.

U prozoru koji sadrži blokovski dizajn otvoriti karticu *Address Editor*. Uočiti osnovne adrese (*Master Base Address*) koje pripadaju sklopovima *AXI GPIO 0* i *AXI GPIO 1*. Ove adrese bit će preko XSA datoteke prenesene u okruženje za razvoj programske podrške.



Slika 8.1 Shema računalnog sustava.

U prozoru *Diagram* pomoću desne tipke miša pokrenuti komandu *Validate Design* i prema potrebi ispraviti greške.

Gotov dizajn pohraniti pomoću komande *File/Save Block Design*.

8.5.2 Generiranje HDL modela pojedinih komponenata

Na način opisan u poglavlju 5.7.2 podesiti VHDL kao ciljani jezik (*Target Language*) te generirati modele pojedinih komponenata.

8.5.3 Generiranje krovnog HDL modela

Na način opisan u poglavlju 5.7.3 generirati krovni model.

8.6 Analiza dizajna

Provesti analizu dizajna na način opisan u poglavlju 3.7. Analizirati dobivenu shemu, provjeriti je li ona u skladu s očekivanjima te po potrebi ispraviti greške.

8.7 Sinteza

Provesti sintezu na način opisan u poglavlju 3.10.

8.8 Definiranje zahtjeva na implementaciju

Datoteka koja sadrži zahtjeve za implementaciju već je kreirana, a zove se *es.xdc*. Njeno kreiranje provedeno je tijekom prethodne vježbe, a način kreiranja opisan je u poglavlju 3.11.1.

Sada je potrebno dodati nove priključke na način opisan u poglavlju 3.11.2. Pokrenuti komande

Flow/Open Synthesized Design
Window/ I/O Ports

Proučiti dobiven prozor *I/O Ports*. Uočiti da su signali koji su bili korišteni na prethodnoj vježbi već spojeni na vanjske priključke. Spojiti nove signale na svjetleće diode LD1 do LD7 te na sklopke BTNL i BTNR. Ne zaboraviti podesiti *I/O Standard*, pritezne otpornike, zaključenja i dr.

Nakon što su priključcima dodijeljeni parametri, potrebno je parametre pohraniti u *.xdc* datoteku.

Otvoriti *.xdc* datoteku i provjeriti njen sadržaj. Uočiti da su u datoteci već od prethodne vježbe definirani zahtjevi za takt. (Ako zbog bilo kojeg razloga nedostaje redak s definicijom takta, potrebno je definirati zahtjeve za takt na način opisan u poglavlju 3.11.3. te pohraniti rezultat u *.xdc* datoteku.)

Ako je u *.xdc* datoteci bilo što mijenjano, potrebno je ponoviti sintezu.

8.9 Implementacija

Provesti implementaciju na način opisan u poglavlju 3.12.

8.10 Generiranje datoteke za programiranje

Provesti generiranje datoteke za programiranje na način opisan u poglavlju 3.13.

8.11 Prijenos konfiguracije sklopovlja u programsko okruženje

Na način opisna u poglavlju 6.14 napraviti novu XSA datoteku. Dodijeliti joj isto ime kao i u prethodnoj vježbi (presnimiti preko stare datoteke). Kod odabira imena provjeriti je li u polju *Export to* ispravno postavljen put mape šeste vježbe, *prezime_ime\v6*.

8.12 Programska podrška

8.12.1 Pokretanje programa Vitis

Pokrenuti Vitis prema uputama u poglavlju 6.15.1. Paziti da on bude pokrenut u ispravnoj radnoj mapi i po potrebi ju promijeniti. U ovom primjeru radna mapa je *prezime_ime\v6*.

8.12.2 Ažuriranje platforme

Ažurirati platformu na način opisan u poglavlju 7.12.2.

8.12.3 Ažuriranje aplikacijskog i sistemskog projekta

Ažurirati aplikacijski i sistemski projekt na način opisan u poglavlju 7.12.3.

8.12.4 Prepravljanje aplikacije

U aplikaciji koja je preuzeta iz prethodne vježbe potrebno je beskonačnu petlju zamijeniti for petljom s 10 prolaza. Nakon ove petlje dodati beskonačnu petlju koja upravlja trćećim svjetlom opisanim u zadatku.

Dopisati C kod koji predstavlja rješenje zadatka. Temeljiti ga na upravljačkom programu **Gpio**. Prema potrebi, poslužiti se primjerima s predavanja.

Ne zaboraviti pohraniti datoteku s C kodom.

8.12.5 Prevođenje programske podrške

Podesiti opcije za povezivanje i punjenje na način opisan u poglavlju 7.12.5. Prevesti programsku podršku prema uputama u poglavlju 6.15.7.

8.13 Uhodavanje na sustavu ZedBoard

Učitati konfiguraciju koja sadrži sklopovlje i programsku podršku u sustav ZedBoard prema uputama u poglavlju 6.16. Nakon pokretanja, sve diode trebale bi raditi na način opisan u zadatku. Provjeriti mijenja li se na odgovarajući način smjer kretanja svjetla pritiskom na tipke BTNL i BTNR.

Provjeriti rad serijske veze prema uputama u poglavlju 7.13.2. Prema potrebi ispraviti greške.

9. Savjeti za rad s alatima

U ovom poglavlju skupljene su bilješke i savjeti korisni za rad s alatima Vivado i Vitis. Dani su u obliku uputa i pojašnjenja.

9.1 Podešavanje opcija alata

Prije početka rada s Vivadom i Vitisom ponekad je potrebno podesiti postavke prikaza, automatskih radnji koje se provode nad kodom i sl.

9.1.1 Vivado

Postavke Vivada podešavaju se u izborniku

Tools/Settings

Preporučuje se podesiti sljedeće postavke

- Ako se pojedini odsječci razvijenog koda namjeravaju koristiti u dokumentaciji koja se piše u editorima teksta, npr. u Wordu, potrebno je u kartici *Tool Settings/Text Editor/Tabs* isključiti opciju *Use tab character*. Također, preporučljivo je podesiti *Tab size* na vrijednost 3.

9.1.2 Vitis

Postavke Vitisa podešavaju se u izborniku

Window/Preferences

Preporučuje se podesiti sljedeće postavke

- Ako se pojedini odsječci razvijenog koda namjeravaju koristiti u dokumentaciji koja se piše u editorima teksta, npr. u Wordu, potrebno je u kartici *Additional/General/Editors/Text Editors* uključiti opciju *Display spaces for tabs*.
- Ovisno o jeziku na kojem se u kodu pišu komentari, potrebno je podesiti opcije u kartici *Additional/General/Editors/Text Editors/Spelling*.

9.2 Imena datoteka i duljina puta

Vivado i Vitis u radu proizvode mnoštvo podmapa s dugim putom. Na nekim inačicama Windows operacijskog sustava ime datoteke zajedno s putom ne smije biti dulje od 256 znakova. Zato radni direktorij treba biti što bliže korijenskom. Nadalje, imena projekata i datoteka neka ne budu duga. Na Linux operacijskom sustavu put smije biti dulji pa se nema problema s duljinama imena datoteka i mapa.

Vivado ne podržava bjeline (*space*) u imenima datoteka.

9.3 Primjer naziva datoteka

Da bi primjeri za predavanja bili uniformni i intuitivni, dizajneri se kod davanja imena pojedinim komponentama projekta pridržavaju nekih konvencija. Te konvencije obično su

definirane unutar svake tvrtke ili grupe dizajnera. U daljnjem tekstu dan je primjer nazivlja pogodnog za vježbe i manje studentske projekte. U njemu su odabrana kratka imena.

9.3.1 Vivado

Primjer naziva datoteka i mapa u Vivado razvojnom okruženju:

- glavna mapa projekta, primjer **p015_display**
  - Naziv glavne mape se sastoji od slova 'p', broja, znaka '_' i riječi koja opisuje sadržaj projekta. Odabrati ovu riječ što kraćom da se putovi koje generiraju Vivado i Vitis ne prijeđu duljinu od 256 znakova.
- *project name*: **p**
  - Uzeto je samo jedno slovo. Ako se svaki projekt radi u zasebnoj glavnoj mapi, nema potrebe razlikovati više projekata pa je i kratko ime zadovoljavajuće.
- blokovski dizajn: **zynq1.bd**
  - Krovni model ponekad se naziva all_design, ali to je ime je dugačko. Asocijativno je ovdje zvati projekt 'zynq'. Broj je dodan da se ostavi mogućnost da u jednom projektu postoji više sklopovskih konfiguracija.
- *wrapper*: **zynq1_wrapper.vhd**
  - Krovni dizajn može ostati ovakav kako ga predlaže sam Vivado. Ime je dobiveno tako da se na blokovski dizajna doda '_wrapper'.
- XDC datoteka: **zynq1_wrapper.xdc**
  - Ovo ime je odabrano jer datoteka opisuje priključke istoimenog *wrapper*-a.
- XSA datoteka: **zynq1.xsa**
  - Vivado predlaže isti naziv kao i *wrapper*. Međutim, bolje je iz imena izbaciti *_wrapper* da se skrati duljina puta. Pored toga, *zynq1* dovoljno dobro opisuje platformu.

9.3.2 Vitis

Primjer naziva datoteka i mapa u Vitis razvojnom okruženju:

- ime platforme (*Platform Project*): **zynq1**
  - Ovo ime i Vitis ponudi ako se XSA zove zynq1.xsa.
- ime aplikacijskog projekta: **sw1**
  - Asocira da je ovo *software* broj 1. Ime nije dugačko, a broj je tu jer možda bude razvijeno nekoliko aplikacija, npr. jedna za svaki od dva procesora.
- ime sistemskog projekta: **sw1_system**
  - Prihvaćeno je ono što je ponudio sustav. Iako ovo ime stoji u hijerarhiji iznad aplikacijskog projekta, ime za aplikacijski i sistemski projekt zadaje se istovremeno, s tim da se prvo zadaje ime aplikacijskog projekta.

9.4 Položaj datoteka

Ako se neka datoteka, npr. jezgra, doda u projekt, treba ju dodati dublje od *.xpr* datoteke tj. dublje od osnovnog direktorija projekta. To je potrebno ako se namjerava direktorij preseliti ili kopirati na drugo mjesto na disku.

9.5 Prijenos konfiguracije sklopovlja u programsko okruženje

Prije pokretanja prijenosa sklopovlja iz Vivada u programsko okruženje (*Export Hardware*) potrebno je zatvoriti *Elaborated Design* i otvoriti *Block Design*.

9.6 Promjena sklopovlja računalnog sustava

Ako se tijekom razvoja sustava promijeni shema sklopovlja, potrebno je iz Vivada napraviti *Export Hardware* kao što je opisano u poglavlju 6.14. Nakon toga je u Vitisu potrebno napraviti korake opisane u daljnjem tekstu.

9.6.1 Ažuriranje platforme

Ažuriranje platforme obuhvaća sljedeće korake:

1. Odabrati *Platform Project* i pomoću desnog gumba miša pokrenuti komandu *Clean Project*.
2. Odabrati *Platform Project* i pomoću desnog gumba miša pokrenuti komandu *Update Hardware Specification*. Paziti da bude odabrana ispravna XSA datoteka. Odabrati *OK*.
3. Desnim gumbom miša odabrati datoteku s konfiguracijom platforme, (datoteka ima nastavak *.spr*) te ju otvoriti pomoću komande *Open*. U domeni *standalone on ps7_cortexa9_0* odabrati *Board Support Package*. Pokrenuti komandu *Modify BSP Settings ...* te u lijevom dijelu prozora koji se pojavio odabrati *standalone*. Podesiti postavke operacijskog sustava na željene vrijednosti. (Tipično je ovdje potrebno podesiti *stdin* i *stdout* u slučajevima kad se koristi UART1. To se izvodi odabirom *ps_uart_1* u polju *Value*.)
3. Odabrati platformu (*Platform Project*) i pomoću desnog gumba miša pokrenuti komandu *Build Project*.

9.6.2 Ažuriranje aplikacijskog i sistemskog projekta

Ažuriranje aplikacijskog i sistemskog projekta obuhvaća sljedeće korake:

1. Odabrati sistemski projekt (*System Project*) te u njemu pronaći i otvoriti datoteku s nastavkom *.sprj*, koja sadrži postavke sistemskog projekta (*System Project Settings*). U prozoru s postavkama, na kartici *General*, pored naziva *Platform*, odabrati gumb '**...**'. U prozoru za promjenu platforme (*Change Platform*) odabrati komandu *Manage* i podesiti platformu iz željene (trenutne) radne mape. Da se izbjegnu greške, preporučuje se ukloniti s popisa nepotrebne platforme. Odabrati *OK* i ponovo *OK*.
2. Generirati novu skriptu za konfiguraciju linkera (*Linker Script*). Odabrati aplikaciju (*Application Project*) u kojoj se generira nova skripta i pomoću desnog gumba na mišu odabrati komandu *Generate Linker Script*. Tijekom ove faze moguće je podesiti veličinu stoga i *heap*-a.
6. Odabrati sistemski projekt (*System Project*) i pomoću desnog gumba miša pokrenuti komandu *Build Project*.

9.7 Premještanje radne mape projekta

Ako zbog nekog razloga treba premjestiti mapu koja sadrži Vitis i Vivado projekt na novu lokaciju na disku, ili kopirati cijeli projekt, potrebno je napraviti radnje koje ispravno uspostavljaju novu radnu okolinu. Ove radnje su opisane u daljnjem tekstu.

9.7.1 Vivado

Nakon premještanja radne mape koja sadrži Vivado projekt, nije neophodno, ali je preporučljivo napraviti sljedeće radnje:

1. Otvoriti premješteni projekt u Vivadu.
2. Napraviti reset cijelog projekta. Reset se izvodi pomoći komande *reset_project* koju treba napisati i pokrenuti u Tcl konzoli.
3. Pokrenuti redom sintezu, implementaciju i izradu datoteke za programiranje FPGA sklopa (*Bitstream*). Ove radnje bit će automatski napravljene pokretanjem komande *Flow/Generate Bitstream*.
4. Napraviti novu XSA datoteku. Izrada XSA datoteke pokreće se komandom *File/Export/Export Hardware*, pri čemu treba uključiti opciju *Include bitstream*.

9.7.2 Vitis

Nakon premještanja radne mape koja sadrži Vitis projekt, potrebno je napraviti iste radnje koje se provode nakon promjene sklopovlja računalnog sustava. Ove radnje opisane su u poglavlju 9.6.

9.8 Kopiranje aplikacijskog projekta u drugo radno okruženje

Aplikacijski odnosno pripadajući sistemski projekt koji je dio jednog Vitis programskog okruženja moguće je kopirati u drugo Vitis programsko okruženje. To se izvodi na sljedeći način:

1. U polaznom Vitis okruženju pokrenuti komandu *File/Export*. Odabrati jedan ili više sistemskih i/ili aplikacijskih projekata koje se želi kopirati. Podesiti ime i mapu u koji će datoteka s projektima biti pohranjena. Preporučuje se ne uključivati radne datoteke (*build folders*) niti poveznice na datoteke (*file links*). Odabrati *OK*.
2. U odredišnom Vitis okruženju pokrenuti komandu *File/Import*. Odabrati opciju *Vitis project exported zip file* i odabrati *Next*. Odabrati željenu arhivu. Označiti u arhivi projekte koje se želi prekopirati te odabrati *Finish*.

9.9 Sinteza i definiranje priključaka

Da bi se mogao otvoriti prozor *I/O Ports* u kojem se definiraju vanjski priključci, potrebno je napraviti i otvoriti sintezu.

9.10 XDC datoteka

Nakon dodjele priključaka svim signalima i pohrane opisa priključaka u XDC datoteku potrebno je tu datoteku otvoriti i provjeriti njen sadržaj. Ako je shema u blokovskom dizajnu više puta mijenjana, pri čemu su brisani neke komponente i njihovi izlazni signali, moguće je da u XDC datoteci zaostanu redci koji opisuju te stare, sada nepostojeće signale. Bez obzira na to, implementacija će biti uspješno napravljena ako nema konflikta. Međutim alat će javiti *Warning* koji govori da su na pojedine priključke spojeni nepostojeći signali. Zato takve retke treba ručno izbrisati iz XDC datoteke.

9.11 Postupak prevođenja programske podrške

Prije nego što je *Platform Project* preveden, u nekim situacijama u njemu može nedostajati datoteka *xparameters.h*. Ova datoteka pojavit će se u projektu ako se nad njim izvrši komanda *Build Project*.

Često se tijekom razvoja programske podrške provodi brisanje pomoću komande *Project/Clear* s uključenom opcijom opciju *Clean all projects*. Nakon toga provodi se *Project/Build All*. Međutim, ova komadna obrađuje projekte abecednim redom. Stoga će alat prijaviti nedostatak datoteke *xparameters.h* ako se *Platform Project* nalazi abecedno iza *System Project*-a. U tom slučaju potrebno je prvo prevesti *Platform Project*. To se izvodi njegovim odabirom i komandom *Build Project* koja se pokreće desnim gumbom miša. (Dva uzastopna poziva komande *Project/Build All* također rješavaju problem).

Zbog navedenog ponašanja alata, izbjegavati brisanje platformskog projekta osim kada je potrebno promijeniti sklopovlje (poglavlje 9.6) ili radni direktorij projekta (poglavlje 9.7).

9.12 Resetiranje projekta

Vivado nema naredbu za resetiranje projekta u izborniku. Resetiranje se pokreće iz TCL konzole pokretanjem komande

reset_project

10. Završne napomene

Opisane vježbe ilustrirale su osnovne korake u razvoju računalnih sustava temeljenih na sustavima na čipu familije Zynq-7000. Zadaci koji su rješavani u pojedinim vježbama bili su jednostavni, ali cjeloviti. U tom smislu oni predstavljaju osnovu da daljnje učenje i rad sa spomenutom platformom.

Neke specifičnosti, kao što je učitavanje programske podrške iz nebrisive memorije ili memorijske kartice nisu obrađeni. Taj dio ostavljen je čitatelju za daljnji samostalan rad.